

RMB: Reward Model Boosting Mitigates Reward Hacking

Jiabin Fan¹
 Dezhi Ye²
 Yongchang Hao¹
 Lili Mou^{1,3}

jiabin@ualberta.ca
 dezhiye@tencent.com
 yongchang1@ualberta.ca
 doublepower.mou@gmail.com

¹ Dept. Computing Science & Alberta Machine Intelligence Institute (Amii), University of Alberta

² Tencent

³ Canada CIFAR AI Chair

Reviewed on OpenReview: <https://openreview.net/forum?id=bIaCQSyb4g>

Abstract

Reinforcement Learning from Human Feedback (RLHF) is a powerful technique for aligning large language models (LLMs) with human preference. However, it often suffers from the reward hacking issue, where policy optimization improves the proxy reward model while actually degrading performance with respect to the true human preference, due to the imperfection of the proxy. To address this, we propose Reward Model Boosting (RMB), a novel approach that enhances the robustness and reliability of the reward signal for RLHF. RMB first trains a set of reward models with a diversity-promoting regularizer. This encourages each model to learn complementary aspects of the reward landscape. Then, RMB learns a lightweight aggregator in the principle of boosting to aggregate the outputs of the diverse reward models into a more accurate and robust reward signal. Our extensive experiments demonstrate that RMB significantly improves reward accuracy on both in-distribution and out-of-distribution datasets, substantially mitigating the reward hacking issue and ultimately improving RLHF performance.¹

1 Introduction

Large language models (LLMs) can be guided to follow human preferences through reinforcement learning from human feedback (RLHF), which utilizes a reward model trained on labeled human preference data to approximate human judgments of output quality (Bai et al., 2022; Ouyang et al., 2022; OpenAI Team, 2023; Meta Team, 2023). By optimizing the LLM (i.e., the policy) towards this reward model, we can significantly improve the performance of the outputs. This approach has been widely adopted in recent LLM development and has demonstrated strong empirical success in aligning model behavior with human preferences (Ouyang et al., 2022; Gemma Team, 2024; Qwen Team, 2024).

However, a learned reward model for RLHF often suffers from an issue known as *overoptimization* or *reward hacking* (Stiennon et al., 2020; Gao et al., 2023; Coste et al., 2024), where policy optimization improves the proxy reward model while actually degrading performance with respect to the true human preference. To address this, several lines of research have emerged, focusing either on improving the Reinforcement Learning (RL) training process via constrained policy optimization (Moskovitz et al., 2024; Zhang et al., 2024; Liu et al., 2024), or on designing more robust reward models that provide a reliable learning signal for RL training (Coste et al., 2024; Yang et al., 2024; Liu et al., 2025). Our work focuses on the latter direction.

One of the reasons for reward hacking in RLHF is that a reward model trained on limited human-preference data is inevitably an imperfect surrogate for the true underlying human preference (Gao et al., 2023). A

¹Our code is released at <https://github.com/MANGA-UOFA/RMB>

natural solution is to construct a more robust reward signal by combining multiple reward models, therefore reducing reliance on any single imperfect proxy. Along this line, Coste et al. (2024) average the predictions of multiple reward models to reduce the noise of individual models. Nevertheless, the effectiveness of such ensemble-based methods is constrained by the fact that reward models trained on similar data and objectives often share systematic blind spots, causing them to “herd” toward the same erroneous judgments (Eisenstein et al., 2024). This suggests that robustness requires not merely aggregating multiple reward models, but explicitly promoting diversity among them and learning how to combine their complementary strengths.

Motivated by these limitations, we propose **Reward Model Boosting (RMB)**, where we construct a more reliable reward signal with advanced boosting methods. First, we train a group of reward models jointly with a diversity-promoting penalty; in particular, we apply the Hilbert–Schmidt Independence Criterion (HSIC; Gretton et al., 2005; 2007), which reduces the correlation among models and promotes complementary error profiles. Then, we train a lightweight aggregator, following the principle of boosting, to combine the reward models effectively. Specifically, we learn a decision tree-based aggregator (Chen & Guestrin, 2016) that combines the reward models’ outputs and iteratively minimizes the aggregated reward prediction error. The boosted decision-tree aggregator exploits reward models’ complementary errors, yielding a more accurate and robust reward signal. Our approach offers two key advantages. (1) *Stronger aggregation*. Our RMB method learns to correct one reward model’s errors using the strengths of others during boosting training, thus producing more accurate predictions than simple averaging (shown in §3.4). (2) *Robustness to text perturbation*. By using a boosted decision-tree aggregator, RMB induces a discretized mapping from model outputs to final scores, effectively smoothing spurious fluctuations caused by training noise (shown in §3.5).

We conduct extensive experiments to validate the effectiveness of our approach. First, we demonstrate that RMB significantly improves reward prediction accuracy on both in-distribution and out-of-distribution (OOD) datasets, indicating strong generalization ability (§3.2). Second, we conduct Best-of- N (BoN) sampling and PPO training experiments, showing that our RMB effectively mitigates the reward hacking issue (§3.3). Third, our in-depth analysis verifies that the HSIC diversity-promoting regularizer diversifies the reward models, which is essential in our reward model boosting framework (§3.4). In addition, we perform text perturbation as a reward model attack, which further demonstrates the robustness of our RMB (§3.5).

2 Method

In this section, we present our RMB in detail. We first present the task formulation in §2.1, and then introduce our boosting method in §2.2. Finally, we discuss diverse reward modeling in §2.3, which is important for performance gain.

2.1 Task Formulation

Let a dataset $\mathcal{D}$ contain tuples of (x, y_w, y_l) , where x is an input text, and y_w and y_l are two responses to x . A human has indicated a preference of y_w over y_l (w for win and l for loss).

Our RMB involves a set of reward models $r_1, r_2, \dots, r_N$, where each model r_i maps an input–response pair (x, y) to a scalar reward score. The reward scores form a vector:

$$\mathbf{R}(x, y) = [r_1(x, y), r_2(x, y), \dots, r_N(x, y)]. \quad (1)$$

Typically, a reward model is trained to assign a higher score to a preferred response y_w than to a disfavored one y_l by minimizing a pairwise comparison logistic loss (Ouyang et al., 2022; Gao et al., 2023; Coste et al., 2024; Yang et al., 2024; Liu et al., 2025). We further incorporate a diversity penalty, deferred to §2.3.

Our boosting objective is to learn an aggregator function F combining this reward vector into a scalar reward score, which is used to evaluate the quality of a response y given its input x for RLHF.

2.2 Boosting the Reward Models

We propose to learn the aggregator function F by adapting the gradient boosting framework, specifically following the principle of XGBoost (Chen & Guestrin, 2016). The key idea is to construct F_K as an ensemble

of K regression decision trees $(f_1, f_2, \dots, f_K)$, where each tree is trained on the residuals (the errors) from the combined predictions of all preceding trees. This sequential, additive modeling allows the aggregator to capture sophisticated, nonlinear interactions among the base reward model outputs.

Formally, the aggregator at iteration k is expressed as:

$$F_k(\mathbf{R}(x, y)) = \sum_{t=1}^k f_t(\mathbf{R}(x, y)), \quad (2)$$

where each f_t is a regression tree that maps the reward vector $\mathbf{R}(x, y)$ to a scalar value. This is accomplished by traversing from the root to a leaf node, guided by internal nodes that contain a split condition based on a feature (an element in $\mathbf{R}(x, y)$) and a threshold (e.g., if $\mathbf{R}(x, y)[1] > 0.5$, go left; otherwise, go right). The final prediction is a scalar score held by a leaf node at the end of the path. The learnable parameters of a regression tree are its internal node split conditions and the scores held by the leaf nodes.

Training objective. Given a dataset $\mathcal{D}$ of pairwise preference annotations, where each training example is a tuple (x, y_w, y_l) , we aim to learn a reward function, modeled by F_K , that assigns a higher score to (x, y_w) than to (x, y_l) . Following boosting, trees are added iteratively so that at iteration k we fit f_k to reduce the residual errors left by F_{k-1} . The training objective at iteration k is

$$\mathcal{L}^{(k)} = \sum_{(x, y_w, y_l) \in \mathcal{D}} L\left(F_k(\mathbf{R}(x, y_w)) - F_k(\mathbf{R}(x, y_l))\right) + \Omega(f_k) \quad (3)$$

$$= \sum_{(x, y_w, y_l) \in \mathcal{D}} L\left(\underbrace{F_{k-1}(\mathbf{R}(x, y_w)) - F_{k-1}(\mathbf{R}(x, y_l))}_{\text{residual errors}} + f_k(\mathbf{R}(x, y_w)) - f_k(\mathbf{R}(x, y_l))\right) + \Omega(f_k). \quad (4)$$

The data-fitting term in Eqn. (3) uses the pairwise logistic loss:

$$L(F_k(\mathbf{R}(x, y_w)) - F_k(\mathbf{R}(x, y_l))) = -\log(\sigma(F_k(\mathbf{R}(x, y_w)) - F_k(\mathbf{R}(x, y_l)))), \quad (5)$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$. This encourages the preferred response y_w to receive a higher score than y_l . The regularizer $\Omega(f_k) = \gamma T_k + \frac{\lambda}{2} \sum_{j=1}^{T_k} w_{kj}^2$ penalizes overly complicated trees and large leaf values, where T_k is the number of leaves in tree f_k , and w_{kj} is the value of leaf j , tree k . The coefficients γ and λ control the strength of the penalty.

Learning the decision trees. The structure is learned through a greedy algorithm, starting from a single root node and iteratively adding branches. At each node, the algorithm searches for an optimal split condition (in the form of feature thresholding). The quality of a potential split is measured by the reduction of the objective function in Eqn. (3), referred to as a gain:

$$\text{Gain} = \mathcal{L}_{\text{before split}}^{(k)} - \mathcal{L}_{\text{after split}}^{(k)}. \quad (6)$$

To compute the gain in a tractable way, we follow Chen & Guestrin (2016) and use a second-order Taylor expansion to approximate the objective function (5). Details are provided in Appendix A.1.

The process of adding tree branches stops when the best candidate gain falls below a threshold; once the tree structure is fixed, each leaf value is set by a closed form that minimizes the second-order approximation of Eqn. (3) with the ℓ_2 penalty in $\Omega(\cdot)$. Likewise, we stop adding trees for boosting when the objective no longer improves with an additional tree.

Overall, our RMB approach iteratively minimizes the objective by discovering new tree structures, each focusing on data samples that are currently misranked. In this way, it progressively refines F_M into an accurate reward model aggregator.

2.3 Learning Diverse Reward Models for Boosting

Boosting works best when base reward models make accurate and complementary predictions. When these reward models' scores are highly correlated, they cannot contribute to loss reduction during the boosting process (Geng et al., 2007; Pan et al., 2009; Tološi & Lengauer, 2011; Yeh & Tsai, 2022; Lyzhin et al., 2023).

A naïve approach to encourage diversity of reward model learning is to introduce stochasticity (e.g., different random seeds) in training (Coste et al., 2024; Eisenstein et al., 2024). However, models trained on the same data tend to converge to similar functions even with different random seeds (Kornblith et al., 2019; Chizat et al., 2019). We therefore explicitly introduce a diversity-promoting regularization term, in addition to the conventional reward model training objective.

Reward model learning. Following Bradley & Terry (1952), the main objective of reward model learning is to distinguish between a preferred response y_w and a less preferred one y_l :

$$\mathcal{L}_{\text{reward}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))], \quad (7)$$

where $r_\theta(x, y)$ represents the reward score for input x and its corresponding response y with model parameters θ , and $\sigma(\cdot)$ is the sigmoid function.

HSIC diversity regularization. We adopt the Hilbert–Schmidt Independence Criterion (HSIC; Gretton et al., 2005; 2007) to quantify and penalize statistical dependence between reward models. A key motivation is that redundancy among reward models is often *nonlinear*: two models may exhibit weak global correlation yet rely on the same shortcut features and thus fail on the same prompt subpopulations. As a kernel-based dependence measure, HSIC is sensitive to a broad class of nonlinear relationships, making it well-suited for encouraging genuinely complementary reward signals. Moreover, HSIC yields a direct dependence penalty that is easy to compute on minibatches and integrates naturally as a differentiable regularizer during training.

Importantly, we do not apply HSIC to raw reward scores $r_\theta(x, y)$, since reward scales and offsets are not semantically identified and directly decorrelating raw scores can conflict with the primary preference-learning objective in Eqn. (7). Instead, we regularize dependence at the level of *preference margins*, which preserves invariance to per-model affine transformations while targeting diversity in the decision-relevant quantity that drives pairwise comparisons.

Let $r_{\theta_1}, \dots, r_{\theta_N}$ be N reward models and $\{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^b$ be a batch of samples. We introduce a margin vector $\mathbf{m}_k$ for the k -th reward model:

$$\mathbf{m}_k = [m_k^{(1)}, \dots, m_k^{(b)}]^\top \quad \text{with} \quad m_k^{(i)} = r_{\theta_k}(x^{(i)}, y_w^{(i)}) - r_{\theta_k}(x^{(i)}, y_l^{(i)}). \quad (8)$$

These margins quantify how strongly model k prefers $y_w^{(i)}$ over $y_l^{(i)}$ for input $x^{(i)}$. If two models yield nearly identical margin vectors on a batch, they are learning essentially the same notion of preference strength, i.e., collapsing to a shared margin distribution. Promoting diversity based on the margin vectors helps to learn diverse reward models in a way that they agree on sign, but differ in agreement strength.

Specifically, HSIC uses a kernel function to create a unique representation for each reward model in a high-dimensional space (Gretton et al., 2005; Ullah et al., 2024; Birjais et al., 2024). HSIC checks whether the pattern of one reward model aligns with another in the high-dimensional space based on pairs of margin vectors, given by $\text{HSIC}_b(\mathbf{m}_i, \mathbf{m}_j)$. Details of this HSIC regularizer computation are provided in Appendix A.2.

In summary, we train all N reward models jointly with the following objective:

$$\mathcal{L}_{\text{total}}(\theta_1, \dots, \theta_N) = \sum_{k=1}^N \mathcal{L}_{\text{reward}}(\theta_k) + \lambda_{\text{HSIC}} \sum_{1 \leq i < j \leq N} \text{HSIC}_b(\mathbf{m}_i, \mathbf{m}_j), \quad (9)$$

where $\lambda_{\text{HSIC}} > 0$ controls the strength of the diversity regularization. Minimizing this joint objective (9) yields an ensemble of reward models that are not only individually accurate but also exhibit diversity in preference prediction.

3 Experiments

In this section, we present the empirical evaluation of our proposed RMB. We begin by describing the datasets, models, and competing approaches, followed by our main results and in-depth analyses.

Approach	Unified-Feedback	HHH-Align	MT-Bench	RewardBench				
				Avg.	Chat	Chat-Hard	Safety	Reasoning
Frozen RM	63.9	68.6	68.2	x	x	x	x	x
Baseline RM	68.8	70.3	69.1	64.5	95.8	37.3	59.9	64.8
Label Smooth	68.5	68.8	71.9	61.1	91.6	39.0	53.8	60.2
Margin Loss	69.6	69.8	71.0	66.4	97.2	37.5	56.8	72.7
GRM	71.5	78.7	73.0	66.8	94.1	41.9	69.5	61.5
Avg. Ensemble ($N = 3$)	69.9	72.2	71.1	65.2	96.1	38.2	58.8	67.6
Avg. Ensemble ($N = 5$)	70.8	74.5	72.6	68.9	95.3	41.7	70.4	70.2
RMB ($N = 3$)	73.6	81.3	76.3	68.2	96.4	44.1	72.4	66.0
RMB ($N = 5$)	74.6	83.2	76.0	71.7	95.3	43.6	78.3	71.4

Table 1: Preference prediction performance on all approaches learned using 40K data mode of Unified-Feedback. Results of competing approaches are directly sourced from Yang et al. (2024). N means the number of reward models used for ensembling or boosting.

3.1 Experimental Settings

Datasets. We evaluate our approach on four human preference datasets commonly used in prior work (Zhang et al., 2026; Mandal et al., 2025b). **Unified-Feedback**²: One of the largest publicly available collections of pairwise human feedback. **HHH-Alignment** (Askell et al., 2021): A preference dataset constructed to evaluate language models on helpfulness, honesty, and harmlessness. **MT-Bench Human Judgments** (Zheng et al., 2023): A collection of human preferences for model responses to questions from the MT-Bench benchmark. **RewardBench** (Lambert et al., 2024): A benchmark created to evaluate the capabilities and safety alignment of reward models.

We use the Unified-Feedback dataset for training and in-distribution evaluation; we treat HHH-Alignment, MT-Bench, and RewardBench as out-of-distribution (OOD) evaluation sets, as their input and response distributions differ significantly from those of the training data. Notice that HHH-Alignment, MT-Bench, and RewardBench contain only 221, 5,755, and 2,985 evaluation samples, respectively, and do not have training samples. Thus, we use them exclusively for OOD evaluation, following previous work (Yang et al., 2024).

Models. Following the setup of Yang et al. (2024), we use Gemma-2B-IT (Gemma Team, 2024) as the backbone for both reward model training and policy optimization; we use a publicly available Mistral-7B-Instruct-v0.2 model³ that was fine-tuned on the Unified-Feedback dataset as the gold reward model serving for a synthetic setup to detect the reward hacking issue.

Competing approaches. We compare the performance of RMB against six competing approaches: **Baseline Reward Model:** It is a standard model trained with the classic reward objective defined in Eqn. (7). **Frozen Reward Model:** Here, the reward model reuses the base language model’s weights, except that a two-layer neural classifier is fine-tuned. **Margin Loss:** An additional margin is incorporated into the reward loss, as shown by Eqn. (31) in Appendix B (Meta Team, 2023; Wang et al., 2024a). **Label Smoothing:** A regularization technique that penalizes overconfident model outputs to mitigate overfitting (Wang et al., 2024a). **GRM:** The GRM method employs a regularization term of the text generation objective during reward model learning (Yang et al., 2024). **Average Ensemble:** A method that averages the outputs of a group of reward models learned in the same way as the baseline reward model with different random seeds as the final reward (Eisenstein et al., 2024). Unless stated otherwise, we average outputs of three reward models as an ensemble.

All competing approaches and RMB are evaluated under the same controlled experimental setup, including identical data splits, preprocessing, backbone models, and training frameworks. For implementation details of all approaches, please refer to Appendix B.

²Unified-Feedback: <https://huggingface.co/datasets/llm-blender/Unified-Feedback>

³Mistral-7B-Instruct-v0.2 model: <https://huggingface.co/Ray2333/reward-model-Mistral-7B-instruct-Unified-Feedback>

Approach	Opponent	Pairwise Preference Comparison (in %)				Pointwise Preference Comparison (in %)			
		Win $\uparrow$	Tie	Lose $\downarrow$	Δ	Win $\uparrow$	Tie	Lose $\downarrow$	Δ
RMB ($N = 3$)	Baseline RM	55.97	3.21	40.82	$\uparrow$ 15.15	44.73	23.97	31.29	$\uparrow$ 13.44
	Avg. Ensemble ($N = 3$)	57.87	4.31	37.81	$\uparrow$ 20.06	40.22	34.50	25.28	$\uparrow$ 14.94
	GRM	54.76	3.01	42.22	$\uparrow$ 12.54	43.33	30.39	26.28	$\uparrow$ 7.05

Table 2: LLM-as-judge of policies learned via RLHF.

3.2 Reward Model Performance

We evaluate the effectiveness of our approach through two main experiments: (1) reward model performance on human-preference data (i.e., whether the learned reward model can rank human preferred responses with a higher score or not), and (2) generation performance of the policy learned by RLHF based on different reward models (Ouyang et al., 2022).

Preference prediction performance. Following the setup of Yang et al. (2024), we learn reward models using Unified-Feedback with two training sizes: 40K samples (Tab. 1 in the main paper) and 400K (Tab. 8 in Appendix C). We compute the accuracy that a reward model correctly ranks the win/lose responses (i.e., assigning a higher reward to the win response). We use the data processing code released by Yang et al. (2024) to ensure a fair comparison.

As seen from the tables, recent methods of learning reward models—such as Label Smooth (Wang et al., 2024a), Margin (Meta Team, 2023), GRM (Yang et al., 2024), Avg Ensemble (Coste et al., 2024)—bring marginal improvement compared with the baseline approach (Ouyang et al., 2022). By contrast, our RMB approach consistently and substantially outperforms all competing methods in both in-distribution and OOD setups. In particular, our model surpasses the classic average ensemble (Coste et al., 2024) when the number of ensemble individuals is controlled ($N = 3$ and $N = 5$ in our experiments).

LLM-as-judge evaluation of policies learned via RLHF. We use an LLM judge to evaluate the policies (i.e., language models) learned by RLHF. Specifically, we use Gemma-2B-IT as a base policy model and fine-tune it by the PPO algorithm using the reward models learned in the previous setup. Then, we prompt DeepSeek-R1 (DeepSeek-AI Team, 2025) to judge the overall quality of the responses for the inputs in the Unified-Feedback hold-out test set. We select three competitors in this evaluation due to the limitation of budget: Baseline RM as a baseline method, GRM, and Avg Ensemble as strong competitors. To mitigate ID and positional biases (Zheng et al., 2024; Shen et al., 2023), we conduct pairwise comparison in a way that each pair is queried four times by swapping the candidate order and their IDs (i.e., “A” and “B”). In addition, we also conduct a pointwise comparison, where we obtain an evaluation score ranging from 1 to 10; then, we compute a win/tie/loss rate according to the score comparison. LLM evaluation prompt templates are provided in Appendix H.

Tab. 2 reports the results of LLM-as-judge. As seen, RMB yields an LLM policy that attains the highest win rate. This is consistent against all competitors and in both pairwise and pointwise settings. These findings, along with the results in the preference prediction experiment, convincingly demonstrate the effectiveness of our RMB method for RLHF.

3.3 Evaluation of the Mitigation of Reward Hacking

We analyze the reward hacking by Best-of-N (BoN) sampling and Proximal Policy Optimization (PPO) experiments, following prior work (Gao et al., 2023). This involves a synthetic data setup, where we use Mistral-7B-Instruct-v0.2 as the gold reward to provide preference annotations for training a reward model (called a proxy reward).

BoN. BoN sampling draws n candidate responses from a general language model (i.e., Gemma-2B-IT) and picks the one with the highest score under a proxy reward model. As n increases, if the chosen output’s proxy score rises while its gold reward score plateaus or even declines, then it shows that this proxy reward model suffers from reward hacking. BoN sampling has been widely adopted as an experiment setup to illustrate the overoptimization issue (Gao et al., 2023; Yang et al., 2024; Liu et al., 2025).

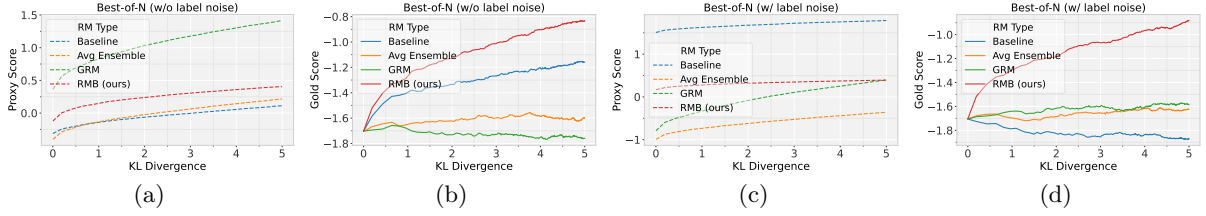


Figure 1: Results of Best-of-N sampling experiments, where both Avg Ensemble and our RMB use three individual reward models. Figures 1a and 1c intend to show the change of the proxy reward score, as an increasing proxy reward with a decreasing/plateaued gold reward indicates reward hacking. Notice that the scores of different proxy reward models are not directly comparable. Figures 1b and 1d show the gold reward scores achieved, a higher score indicating a better reward model.

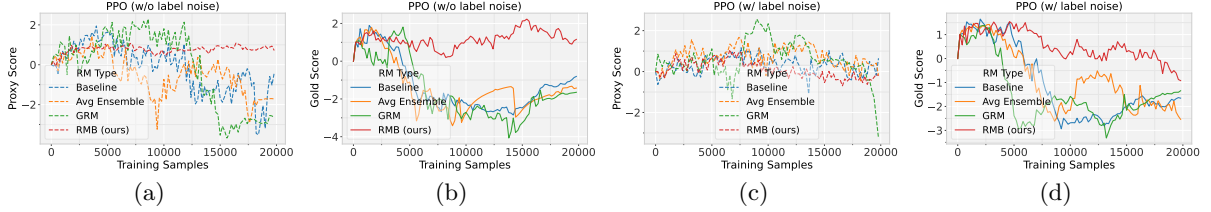


Figure 2: PPO training with reward models (RMs). Ensemble and RMB both have three RMs to aggregate.

Our experiment adopts the setup in Yang et al. (2024). We first draw a subset containing 20K samples from Unified-Feedback and annotate preference labels (i.e., which sample is preferred) based on the gold reward to train a proxy reward model. For each input in the test set, we perform BoN sampling (i.e., drawing n candidates and ranking them with the proxy model). The top-ranked response is selected and then scored by the gold reward. We report both the average proxy score and the average gold score across the test inputs. We vary the selection budget via the BoN KL divergence from 0 to 5, corresponding to the number of responses n ranging from 1 to 405 per input. The KL divergence is given by $KL_{BoN} = \log n - \frac{n-1}{n}$, which quantifies how n -category uniform distribution deviates from the one-hot distribution of a sample.

Figs. 1a and 1b show the performance of our RMB approach and competing approaches in the BoN sampling experiment. We see that, for competing approaches, the gold reward score increases much less than the proxy reward score as KL grows, indicating reward hacking exists. By contrast, RMB exhibits a clear, monotonic increase in the gold reward score with larger KL, demonstrating its advantage in mitigating reward hacking.

It is noticed that human-preference data typically contains $\sim 20\text{--}30\%$ label noise (Wang et al., 2024a), which may degrade reward model generalization (Ramé et al., 2024; Liang et al., 2024) and hinder policy learning (Ye et al., 2023; Mandal et al., 2025a). To assess robustness under label noise, we randomly choose 25% of the data samples and flip their gold reward-induced preference labels.

The results in Figs. 1c and 1d show that, with label noise, the baseline suffers from significant reward hacking, as its gold reward score decreases. GRM and Avg Ensemble demonstrate some capacity of mitigating reward hacking, but the gold reward score achieved still plateaus as their proxy reward scores continue to increase. By contrast, RMB achieves the highest gold reward score, which increases robustly. This indicates that our approach can reliably estimate the response quality even when trained on noisy preference labels.

PPO. PPO (Schulman et al., 2017) is a classic RL algorithm designed to learn a policy maximizing expected rewards. It has been widely adopted in RLHF to learn a language model policy maximizing the preference score provided by reward models (Ouyang et al., 2022; OpenAI Team, 2023; Liu et al., 2024) and also serves as a standard testbed for studying the reward hacking issue (Gao et al., 2023; Moskowitz et al., 2024; Bukharin et al., 2025).

Specifically, we adopt the Gemma-2B-IT model as an initial policy, and use the PPO algorithm to optimize this policy based on the proxy reward model (which is learned by gold reward-induced preference labels). The

		Unified-Feedback (In-Distribution)					RewardBench (Out-of-Distribution)				
Random Seeds	Individual RMs	70.71	71.34	70.52	69.85	70.30	68.45	67.66	69.97	67.11	67.24
	Avg Ensemble	70.80					68.92				
	Our RMB	71.04					68.70				
HSIC	Individual RMs	73.40	70.73	69.29	68.05	73.84	70.08	69.95	67.58	66.97	71.62
	Avg Ensemble	72.04					69.40				
	Our RMB	74.58					71.65				

Table 3: Ensemble vs Boosting with different diversity strategies. We report the performance of individual RMs (index from 0 to 4).

PPO-finetuned policy is then used to generate responses on the inputs from the test set of Unified-Feedback. We track the changes of both proxy and gold reward scores of the generated response throughout PPO training to assess whether improvement under the proxy reward score translates to that under the gold reward score.

As seen from Figs. 2a and 2b, all methods increase the proxy reward score at the beginning of training; however, competing approaches exhibit noisier trajectories with a noticeable mid-training drop, whereas RMB shows smoother and more stable improvements. For the gold score, competing approaches improve briefly only in the first few steps and collapse later. On the contrary, the curve of the gold reward in our approach is more reflected by the proxy reward curve. The results suggest that our approach is able to substantially mitigate reward hacking.

As in the BoN experiment, we also have a setup with 25% noise added to preference labels in the PPO experiment, and the learning curves are shown in Figs. 2c and 2d. As expected, the gold reward score is more fluctuated due to the added label noise. But still, our RMB maintains the highest and least decreasing gold score, demonstrating that our approach is more robust than competing methods with noisy preference labels.

3.4 Analysis of the Components of RMB

We perform an in-depth analysis of two main components of our approach, namely, the boosting method and the HSIC diversity-promoting regularization. We follow most of the setups in §3.2, except that we consider five⁴ individual reward models (RMs) by either varying random seeds or using our HSIC diversity-promoting regularizer.

As shown in Tab. 3, Average Ensemble and RMB achieve similar performance in the random seed setup, whereas with HSIC-induced diversity, our boosting considerably outperforms Average Ensemble. This shows that different random seeds yield similar reward models, thus making ensembles less effective. By contrast, our HSIC encourages complementary error profiles that a boosting aggregator can leverage to achieve substantially higher performance.

The correlation among RMs can be further viewed from the heatmaps in Fig. 3. For each pair of RMs, we measure the ratio of pairwise-preference agreement on the Unified-Feedback test set. RMs learned under random seed diversity exhibit very high pairwise agreement (most pairs >80%), indicating strong correlations; HSIC-trained RMs are less correlated (typically 50–80%), providing the diversity that boosting needs.

We further examine the contribution of each RM during the boosting process. This is measured by *Total Gain* (Chen & Guestrin, 2016), i.e., the sum of gains over all splits (see Eqn. (6)) that each RM obtains in the decision trees in Tab. 4. We see that, under the random seeds setup, Total Gain is heavily skewed (e.g., a single RM contributes 70.13%). This is because such RMs tend to be redundant and thus are ineffective when the boosting process minimizes residual errors.

Under the HSIC setup, the Total Gain is distributed more evenly. RM 0, RM 4, and RM 1 each contribute 25–30%, and they are the strong individual RMs (seen in Tab. 3). This shows that boosting emphasizes the

⁴In the main experiment, we try different numbers of individuals ($N = 3, 5, 8, 10$) for our RMB, but compare our approach with Avg Ensemble in the setting of $N = 3$ as we quote results from previous work for Tab. 1. Here, we consider five individual RMs to have a more reliable correlation analysis.

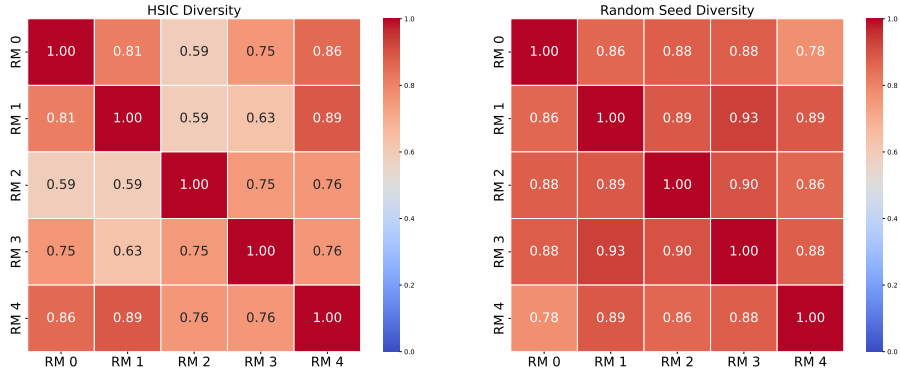


Figure 3: Correlation heat map among individual Reward Models

HSIC Diversity		Random Seeds Diversity	
Model	Total Gain	Model	Total Gain
RM 0	29.75%	RM 1	70.13%
RM 4	27.39%	RM 0	12.39%
RM 1	26.78%	RM 4	10.99%
RM 2	8.55%	RM 2	5.82%
RM 3	7.53%	RM 3	0.66%

Table 4: Total Gain ratio of individual RMs.

best RMs while using the remaining RMs to correct residual errors. Overall, our boosting approach surpasses every individual RM.

Discussion. The low correlation of individuals is shown to be important for boosting algorithms in commercial ranking systems, such as search and recommendation (Covington et al., 2016; Wang et al., 2022; Xi et al., 2023), for which researchers perform heavy feature engineering and propose heterogeneous signals (e.g., semantic relevance, like count, and comment count) to be considered during boosting (Ni et al., 2021; Halder et al., 2023; Du et al., 2024). In our setting, HSIC serves as an auxiliary objective that encourages low pairwise correlation across RMs, which is analogous to the feature engineering process of traditional boosting systems.

3.5 Analysis of Robustness under Attacks

We evaluate RM robustness under small, meaning-preserving text perturbations. This is accomplished by comparing how an RM ranks a set of response candidates for the same input, with and without perturbation. Specifically, we sample 400 candidate responses for each input in the test set of Unified-Feedback using a pretrained Gemma-2B-IT. We first compute reward scores on the original texts and obtain a ranking in descending order for each input. We then apply a keyboard-neighbor typo attack to the text by randomly replacing characters with adjacent keys on the QWERTY layout (Belinkov & Bisk, 2018; Pruthi et al., 2019), and compute a new rank. To study sensitivity, we vary the perturbation budget (number of character edits per example) in $\{1, 3, 5, 8, 10\}$.

We measure the similarity of the two rankings by three metrics: (1) **Pairwise accuracy**: the ratio of correctly ordered pairs among all pairs, (2) **Footrule similarity**: one minus normalized Spearman’s footrule distance, which sums the absolute differences between the ranks of each item in the two rankings (Diaconis & Graham, 1977), and (3) **Normalized Discounted Cumulative Gain (nDCG)**: a metric that measures the usefulness of items based on their position in the rank (Järvelin & Kekäläinen, 2002), where we treat the original order as the ground truth. All metrics are in the range of $[0, 1]$, and the higher, the better.

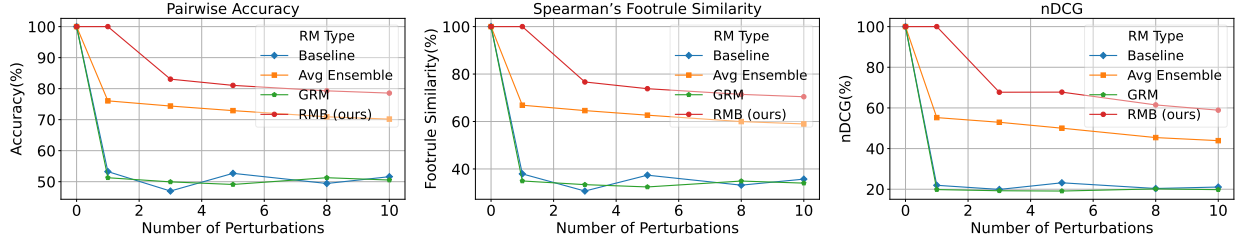


Figure 4: Rank order accuracy.

Latency Type	Component/Model	Mean	p50	p90	p99
Breakdown (s/batch)	Base RMs	19.7915	19.7442	20.2839	20.3000
	Avg Ensemble aggregator	5.92×10^{-5}	5.67×10^{-5}	6.09×10^{-5}	1.398×10^{-4}
	RMB aggregator	1.7458×10^{-3}	1.6997×10^{-3}	1.9010×10^{-3}	2.5694×10^{-3}
End-to-End (s/batch)	Avg Ensemble	19.7915	19.7443	20.2840	20.3001
	RMB	19.7932	19.7460	20.2857	20.3016

Table 5: Inference-efficiency summary.

Figure 4 shows the results of our RMB ($N = 3$), along with the Baseline RM, Avg Ensemble ($N = 3$), and GRM. As seen, our RMB attains the highest scores on all three metrics, indicating that our RMB is more robust than competing approaches under small, meaning-preserving attacks.

Discussion. Decision-tree-based aggregators are known to be robust in ranking tasks (Joachims et al., 2017; Wang et al., 2018; Haldar et al., 2019; Wu et al., 2022), which partially explains that our RMB can maintain high performance during attacks. It is interesting to notice that a Transformer (Vaswani et al., 2017)-based reward model maps discrete text to continuous reward values, and a small change of the input text may lead to large changes of the output, especially when the training set is small (Wu et al., 2025); such a phenomenon is observed in other tasks as well (Ebrahimi et al., 2018; Jin et al., 2020). Our RMB performs aggregation of individual RMs by only predicting one of its leaf values, which are a finite discrete set. This reduces spurious fluctuations caused by training noise, making our model more robust under attacks.

3.6 Efficiency Analysis

Our RMB method introduces a lightweight aggregator on top of a set of base RMs. To verify its efficiency, we evaluate its impact on inference latency. Specifically, we decompose the inference process into two stages: (1) the **base RM pass**, which involves a forward pass through each of the three RMs to generate individual scores, and (2) the **aggregation step**, where individual RM scores are combined into a final prediction. We measure the time spent in each stage for both our RMB aggregator and Average Ensemble (Coste et al., 2024). Metrics include

- **Breakdown latency** (s/batch): The wall-clock time required for a given stage. We report the mean, median (p50), 90th (p90), and 99th (p99) percentile latencies across all batches.
- **End-to-end latency** (s/batch): The overall latency including RMs and the aggregator, as well as other code components (such as IO).

The results are summarized in Tab. 5. As seen, the main computation is devoted to the base RM pass, with a mean latency of 19.791 seconds per batch. In comparison, the Average Ensemble aggregator adds a mean overhead of only 5.92×10^{-5} s (≈ 0.06 ms). Similarly, our RMB aggregator is also efficient as the overhead is magnitudes lower than the base RM pass, as the aggregator vs. end-to-end ratio is only 0.000088. The results show that our RMB is a lightweight aggregator. It introduces a negligible latency to the reward model inference. Thus, it also introduces a negligible latency to RLHF training when the rest of the algorithm and implementation are controlled.

Reward Model	Total Params.	Average	Chat	Chat-Hard	Safety	Reasoning
RMB (N=3)	6B	87.8	98.9	75.6	87.3	89.1
GPT-4o-2024-08-06	-	86.7	96.1	76.1	88.1	86.6
GPT-4-0125-preview	-	86.0	95.3	74.3	87.6	86.9
FsfairX-LLaMA3-RM	8B	84.4	99.4	65.1	86.8	86.4
Starling-RM	34B	82.7	96.9	57.2	88.2	88.5

Table 6: Results of different reward models in a budget-constrained scenario.

3.7 Results in Computational Budget-Constrained Scenario

We further examine whether the gains of RMB hold when the computational budget is limited. In this setting, we compare boosting with N reward models against training a single larger RM that has a similar or larger number of parameters. We perform full-parameter fine-tuning for one epoch using the open-source dataset mixture of HH-RLHF, SHP, UltraFeedback, and Summarization.⁵ This mixture was introduced by Dong et al. (2024), who train FsfairX-LLaMA3-RM-8B on it. To ensure that our results are directly comparable, we use the same training data and evaluate on RewardBench as the test set.

We also include stronger competing reward models whose parameter counts are much larger than ours, including a 34B reward model⁶ and GPT-4/GPT-4o used as judges. Tab. 6 shows that RMB performs well in this budget-constrained scenario: it outperforms the 34B reward model and also surpasses GPT-4/GPT-4o as judges. In addition, RMB yields a clear improvement over the 8B baseline, raising the score from 84.4 to 87.8, while using an ensemble with fewer total parameters and the same training data. Overall, these results confirm that RMB remains effective when computational resources are constrained.

4 Related Work

The challenge of reward hacking in RLHF was empirically established by a number of studies (Ouyang et al., 2022; Gao et al., 2023). There are generally two directions to mitigate the reward hacking issue: (i) improving the RL training process under an imperfect reward signal, and (ii) learning a better reward model to provide a reliable training signal for RL.

Optimizing the RL process. One line of work mitigates reward hacking by refining the RL optimization process under imperfect reward signals. For example, researchers update the policy conservatively by designing pessimistic reward signals given an imperfect RM (Zhai et al., 2023; Zhang et al., 2024; Yan et al., 2024; Mandal et al., 2025b). Another strategy is to maintain a pretrained policy’s text generation ability by regularizing the RL objective (Ouyang et al., 2022; Liu et al., 2024; Laidlaw et al., 2025), or averaging policies’ parameters (Lin et al., 2024). Researchers also constrain the policy optimization by restricting the exploration region (Moskovitz et al., 2024; Dai et al., 2025; Mandal et al., 2025a).

Improving reward models. Our work falls into this category, which combats reward hacking by strengthening reward models. Building an ensemble of multiple RMs can stabilize noisy reward signals; for example, Coste et al. (2024) average RM outputs and Ramé et al. (2024) average RM parameters. However, simple averaging cannot remove the bias shared among individual RMs (Eisenstein et al., 2024). To address this, our RMB learns an aggregator based on the principle of boosting (Chen & Guestrin, 2016), which reduces residual reward error in a complementary way.

Most other RM-centric studies focus on improving an individual reward model, including designing better RM training objectives (Yang et al., 2024; Wang et al., 2024b), optimizing RM architectures (Miao et al., 2024; Chen et al., 2024), and improving training data quality (Zhu et al., 2024; Wolf et al., 2025; Haider et al., 2025; Liu et al., 2025; Bukharin et al., 2025). These approaches focus on a single RM, which can be potentially combined with our boosting approach for further performance improvement.

⁵preference-700K: <https://huggingface.co/datasets/hendrydong/preference-700K>

⁶Starling-RM-34B: <https://huggingface.co/Nexusflow/Starling-RM-34B>

Multi-objective alignment. Another related line of work studies alignment with multiple preference objectives, where optimizing one objective may interfere with others. Representative approaches address such conflicts during policy optimization by adaptively combining objective-specific gradients (Li et al., 2025; Chen et al., 2026), while recent work also studies conflicts arising from preference data (Xu et al., 2026). This line of research mainly concerns how multiple objectives can be jointly optimized without undesirable trade-offs or collapse. RMB instead focuses on an earlier stage of the RLHF pipeline: constructing a reliable reward signal before policy optimization. Rather than assigning individual RMs to predefined objectives, RMB applies diversity regularization to encourage different RMs trained on the same preference data to capture latent complementary preference patterns, and then uses boosting to aggregate them into a robust reward signal. Therefore, the two directions largely address orthogonal components of alignment and are difficult to compare directly. An interesting future direction is to investigate whether the complementary patterns learned by RMB correspond to interpretable preference dimensions, such as helpfulness, harmlessness, or honesty, and whether such reward representations can further support downstream multi-objective alignment.

In short, RLHF is a promising research direction, and various methods have been proposed to improve RLHF at different stages. Due to the scope of this paper, our empirical comparisons focus on methods that directly compete with RMB at the reward-modeling stage.

5 Conclusion

To address the challenge of reward hacking in RLHF, we propose Reward Model Boosting (RMB). Our method learns a decision-tree aggregator over an ensemble of reward models to produce a more accurate and robust reward signal. We further adopt the HSIC diversity-promoting regularizer for reward model training. Our experimental results show that RMB enhances preference prediction accuracy across both in-distribution and out-of-distribution datasets. Moreover, our RMB effectively mitigates reward hacking and stays robust when countering text perturbation. Overall, RMB provides a reliable reward signal for training policies to align with human preferences.

6 Limitations

RMB relies on each reward model providing complementary preference signals. In our experiments, the marginal gain diminishes as the number of reward models increases, with performance largely saturating beyond a moderate aggregation size. One possible explanation is that the training data do not contain sufficiently rich and independent learning signals: as more reward models are added, they increasingly learn redundant preference patterns, limiting the benefits of diversity regularization and boosting. Since our experiments are constrained by the coverage and quality of currently available open-source preference datasets, we cannot fully evaluate RMB’s scaling potential under richer and more heterogeneous feedback. Evaluating RMB on larger, higher-quality, and more diverse preference corpora is therefore an important direction for future work.

Another limitation is that the boosting stage may not preserve preference order if the RMs are added with a constant offset, due to the unidentifiability of the Bradley–Terry objective. However, this is less of a concern in our approach, because we first train RMs, based on which the boosting model is then trained. In this way, the boosting model can account for the offset of RMs and adapt accordingly. Nevertheless, we conduct an analysis on the sensitivity of our RMB under reward shifts in Appendix C.4. We observe that injecting the constant into RMB results in only small changes to its pairwise rankings, indicating RMB’s rankings remain largely stable if RMs are shifted.

Acknowledgments

We thank all reviewers and editors for their insightful feedback and constructive comments. This research was supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC), the Amii Fellow Program, the Canada CIFAR AI Chair Program, and the Digital Research Alliance of Canada.

References

- Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Jackson Kernion, Kamal Ndousse, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, and Jared Kaplan. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*, 2021.
- Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- Yonatan Belinkov and Yonatan Bisk. Synthetic and natural noise both break neural machine translation. In *International Conference on Learning Representations*, 2018.
- Roshan Birjais, Kevin I-Kai Wang, and Waleed Abdulla. Training deep neural networks with HSIC and backpropagation. In *Asia Pacific Signal and Information Processing Association Annual Summit and Conference*, 2024.
- Ralph Allan Bradley and Milton E Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39:324–345, 1952.
- Peter Bühlmann and Bin Yu. Analyzing bagging. *The Annals of Statistics*, 30(4):927–961, 2002.
- Alexander Bukharin, Haifeng Qian, Shengyang Sun, Adithya Renduchintala, Soumye Singhal, Zhilin Wang, Oleksii Kuchaiev, Olivier Delalleau, and Tuo Zhao. Adversarial training of reward models. In *Conference on Language Modeling*, 2025.
- Lichang Chen, Chen Zhu, Jiuhai Chen, Davit Soselia, Tianyi Zhou, Tom Goldstein, Heng Huang, Mohammad Shoeybi, and Bryan Catanzaro. Odin: Disentangled reward mitigates hacking in RLHF. In *Proceedings of International Conference on Machine Learning*, pp. 7935–7952, 2024.
- Peter Chen, Xiaopeng Li, Xi Chen, and Tianyi Lin. Reward-free alignment for conflicting objectives. In *International Conference on Learning Representations*, 2026.
- Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- Lenaic Chizat, Edouard Oyallon, and Francis Bach. On lazy training in differentiable programming. In *Advances in Neural Information Processing Systems*, 2019.
- Thomas Coste, Usman Anwar, Robert Kirk, and David Krueger. Reward model ensembles help mitigate overoptimization. In *International Conference on Learning Representations*, 2024.
- Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for YouTube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, pp. 191–198, 2016.
- Juntao Dai, Taiye Chen, Yaodong Yang, Qian Zheng, and Gang Pan. Mitigating reward over-optimization in RLHF via behavior-supported regularization. In *International Conference on Learning Representations*, 2025.
- DeepSeek-AI Team. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Persi Diaconis and Ronald L Graham. Spearman’s footrule as a measure of disarray. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 39(2):262–268, 1977.

- Hanze Dong, Wei Xiong, Bo Pang, Haoxiang Wang, Han Zhao, Yingbo Zhou, Nan Jiang, Doyen Sahoo, Caiming Xiong, and Tong Zhang. RLHF workflow: From reward modeling to online RLHF. *Transactions on Machine Learning Research*, 2024.
- Ruijiang Dong, Feng Liu, Haoang Chi, Tongliang Liu, Mingming Gong, Gang Niu, Masashi Sugiyama, and Bo Han. Diversity-enhancing generative network for few-shot hypothesis adaptation. In *International Conference on Machine Learning*, pp. 8260–8275, 2023.
- Allan Donner and Neil Klar. *Design and Analysis of Cluster Randomization Trials in Health Research*. Arnold London, 2000.
- Yingpeng Du, Ziyang Wang, Zhu Sun, Yining Ma, Hongzhi Liu, and Jie Zhang. Disentangled multi-interest representation learning for sequential recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 677–688, 2024.
- Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. Hotflip: White-box adversarial examples for text classification. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 31–36, 2018.
- Jacob Eisenstein, Chirag Nagpal, Alekh Agarwal, Ahmad Beirami, Alex D’Amour, DJ Dvijotham, Adam Fisch, Katherine Heller, Stephen Pfohl, Deepak Ramachandran, Peter Shaw, and Jonathan Berant. Helping or Herding? Reward model ensembles mitigate but do not eliminate reward hacking. In *Conference on Language Modeling*, 2024.
- Jiabin Fan, Guoqing Luo, Michael Bowling, and Lili Mou. Ketchup: K-step return estimation for sequential knowledge distillation. In *Findings of the Association for Computational Linguistics: EACL 2026*, pp. 778–796, 2026.
- Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proceedings of the International Conference on Machine Learning*, pp. 10835–10866, 2023.
- Gemma Team. Gemma: Open models based on Gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- Xiubo Geng, Tie-Yan Liu, Tao Qin, and Hang Li. Feature selection for ranking. In *Proceedings of the International Conference on Research and Development in Information Retrieval*, pp. 407–414, 2007.
- Arthur Gretton, Olivier Bousquet, Alex Smola, and Bernhard Schölkopf. Measuring statistical dependence with Hilbert-Schmidt norms. In *International Conference on Algorithmic Learning Theory*, pp. 63–77, 2005.
- Arthur Gretton, Kenji Fukumizu, Choon Teo, Le Song, Bernhard Schölkopf, and Alex Smola. A kernel statistical test of independence. In *Advances in Neural Information Processing Systems*, 2007.
- Zafaryab Haider, Md Hafizur Rahman, Vijay Devabhaktuni, Shane Moeykens, and Prabuddha Chakraborty. A framework for mitigating malicious RLHF feedback in LLM training using consensus based reward. *Scientific Reports*, 15(9177), 2025.
- Malay Haldar, Mustafa Abdool, Prashant Ramanathan, Tao Xu, Shulin Yang, Huizhong Duan, Qing Zhang, Nick Barrow-Williams, Bradley C Turnbull, Brendan M Collins, and Thomas Legrand. Applying deep learning to airbnb search. In *Proceedings of International Conference on Knowledge Discovery and Data Mining*, pp. 1927–1935, 2019.
- Malay Haldar, Mustafa Abdool, Liwei He, Dillon Davis, Huiji Gao, and Sanjeev Katariya. Learning to rank diversely at airbnb. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pp. 4609–4615, 2023.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

- Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.
- Di Jin, Zhijing Jin, Joey Tianyi Zhou, and Peter Szolovits. Is BERT really robust? A strong baseline for natural language attack on text classification and entailment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 8018–8025, 2020.
- Thorsten Joachims, Adith Swaminathan, and Tobias Schnabel. Unbiased learning-to-rank with biased feedback. In *Proceedings of the International Conference on Web Search and Data Mining*, pp. 781–789, 2017.
- Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *International Conference on Machine Learning*, pp. 3519–3529, 2019.
- Anders Krogh and Jesper Vedelsby. Neural network ensembles, cross validation, and active learning. In *Advances in Neural Information Processing Systems*, 1994.
- Alex Kulesza and Ben Taskar. Determinantal point processes for machine learning. *Foundations and Trends in Machine Learning*, 5(2–3):123–286, 2012.
- Ludmila I Kuncheva. *Combining Pattern Classifiers: Methods and Algorithms*. John Wiley & Sons, 2014.
- Cassidy Laidlaw, Shivam Singhal, and Anca Dragan. Correlated proxies: A new definition and improved mitigation for reward hacking. In *International Conference on Learning Representations*, 2025.
- Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. Rewardbench: Evaluating reward models for language modeling. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 1755–1797, 2024.
- Chengao Li, Hanyu Zhang, Yunkun Xu, Hongyan Xue, Xiang Ao, and Qing He. Gradient-adaptive policy optimization: Towards multi-objective alignment of large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pp. 11214–11232, 2025.
- Depeng Li, Tianqi Wang, Junwei Chen, Qining Ren, Kenji Kawaguchi, and Zhigang Zeng. Towards continual learning desiderata via HSIC-bottleneck orthogonalization and equiangular embedding. *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 13464–13473, 2024.
- Xize Liang, Chao Chen, Jie Wang, Yue Wu, Zhihang Fu, Zhihao Shi, Feng Wu, and Jieping Ye. Robust preference optimization with provable noise tolerance for LLMs. *arXiv preprint arXiv:2404.04102*, 2024.
- Yong Lin, Hangyu Lin, Wei Xiong, Shizhe Diao, Jianmeng Liu, Jipeng Zhang, Rui Pan, Haoxiang Wang, Wenbin Hu, Hanning Zhang, Hanze Dong, Renjie Pi, Han Zhao, Nan Jiang, Heng Ji, Yuan Yao, and Tong Zhang. Mitigating the alignment tax of RLHF. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 580–606, 2024.
- Tianqi Liu, Wei Xiong, Jie Ren, Lichang Chen, Junru Wu, Rishabh Joshi, Yang Gao, Jiaming Shen, Zhen Qin, Tianhe Yu, Daniel Sohn, Anastasiia Makarova, Jeremiah Liu, Yuan Liu, Bilal Piot, Abe Ittycheriah, Aviral Kumar, and Mohammad Saleh. RRM: Robust reward model training mitigates reward hacking. In *International Conference on Learning Representations*, 2025.
- Yong Liu and Xin Yao. Ensemble learning via negative correlation. *Neural Networks*, 12(10):1399–1404, 1999.
- Zhihan Liu, Miao Lu, Shenao Zhang, Boyi Liu, Hongyi Guo, Yingxiang Yang, Jose Blanchet, and Zhaoran Wang. Provably mitigating overoptimization in RLHF: Your SFT loss is implicitly an adversarial regularizer. In *Advances in Neural Information Processing Systems*, pp. 138663–138697, 2024.
- Ivan Lyzhin, Aleksei Ustimenko, Andrey Gulin, and Liudmila Prokhorenkova. Which tricks are important for learning to rank? In *Proceedings of the International Conference on Machine Learning*, pp. 23264–23278, 2023.

- Debmalya Mandal, Andi Nika, Parameswaran Kamalaruban, Adish Singla, and Goran Radanovic. Corruption robust offline reinforcement learning with human feedback. In *International Conference on Artificial Intelligence and Statistics*, pp. 4429–4437, 2025a.
- Debmalya Mandal, Paulius Sasnauskas, and Goran Radanovic. Distributionally robust reinforcement learning with human feedback. In *Proceedings of the International Conference on Machine Learning*, 2025b.
- Meta Team. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Yuchun Miao, Sen Zhang, Liang Ding, Rong Bao, Lefei Zhang, and Dacheng Tao. InfoRM: Mitigating reward hacking in RLHF via information-theoretic reward modeling. In *Advances in Neural Information Processing Systems*, pp. 134387–134429, 2024.
- Ted Moskovitz, Aaditya K Singh, DJ Strouse, Tuomas Sandholm, Ruslan Salakhutdinov, Anca Dragan, and Stephen McAleer. Confronting reward model overoptimization with constrained RLHF. In *International Conference on Learning Representations*, 2024.
- Xiuyan Ni, Shujian Bu, Lucas Adams, and Igor L Markov. Prioritizing original news on facebook. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, pp. 4046–4054, 2021.
- OpenAI Team. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.
- Feng Pan, Tim Converse, David Ahn, Franco Salvetti, and Gianluca Donato. Feature selection for ranking using boosted trees. In *Proceedings of the Conference on Information and Knowledge Management*, pp. 2025–2028, 2009.
- Danish Pruthi, Bhuwan Dhingra, and Zachary C Lipton. Combating adversarial misspellings with robust word recognition. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 5582–5591, 2019.
- Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Alexandre Ramé, Nino Vieillard, Léonard Hussenot, Robert Dadashi, Geoffrey Cideron, Olivier Bachem, and Johan Ferret. Warm: On the benefits of weight averaged reward models. *arXiv preprint arXiv:2401.12187*, 2024.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Chenhui Shen, Liying Cheng, Xuan-Phi Nguyen, Yang You, and Lidong Bing. Large language models are not yet human-level evaluators for abstractive summarization. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 4215–4233, 2023.
- Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, 2020.
- Laura Toloşi and Thomas Lengauer. Classification with correlated features: Unreliability of feature ranking and solutions. *Bioinformatics*, 27:1986–1994, 2011.
- Farhan Ullah, Irfan Ullah, Rehan Ullah Khan, Salabat Khan, Khalil Khan, and Giovanni Pau. Conventional to deep ensemble methods for hyperspectral image classification: A comprehensive survey. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, pp. 3878–3916, 2024.

- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- Binghai Wang, Rui Zheng, Lu Chen, Yan Liu, Shihan Dou, Caishuang Huang, Wei Shen, Senjie Jin, Enyu Zhou, Chenyu Shi, Songyang Gao, Nuo Xu, Yuhao Zhou, Xiaoran Fan, Zhiheng Xi, Jun Zhao, Xiao Wang, Tao Ji, Hang Yan, Lixing Shen, Zhan Chen, Tao Gui, Qi Zhang, Xipeng Qiu, Xuanjing Huang, Zuxuan Wu, and Yu-Gang Jiang. Secrets of RLHF in large language models Part II: Reward modeling. *arXiv preprint arXiv:2401.06080*, 2024a.
- Dong Wang, Shaoguang Yan, Yunqing Xia, Kavé Salamatian, Weiwei Deng, and Qi Zhang. Learning supplementary NLP features for CTR prediction in sponsored search. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 4010–4020, 2022.
- Haoxiang Wang, Wei Xiong, Tengyang Xie, Han Zhao, and Tong Zhang. Interpretable preferences via multi-objective reward modeling and mixture-of-experts. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 10582–10592, 2024b.
- Xuanhui Wang, Nadav Golbandi, Michael Bendersky, Donald Metzler, and Marc Najork. Position bias estimation for unbiased learning to rank in personal search. In *Proceedings of International Conference on Web Search and Data Mining*, pp. 610–618, 2018.
- Lorenz Wolf, Robert Kirk, and Mirco Musolesi. Reward model overoptimisation in iterated RLHF. *arXiv preprint arXiv:2505.18126*, 2025.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, pp. 38–45, 2020.
- Chen Wu, Ruqing Zhang, Jiafeng Guo, Yixing Fan, and Xueqi Cheng. Are neural ranking models robust? *ACM Transactions on Information Systems*, 41(2):1–36, 2022.
- Zhaofeng Wu, Michihiro Yasunaga, Andrew Cohen, Yoon Kim, Asli Celikyilmaz, and Marjan Ghazvininejad. reWordBench: Benchmarking and improving the robustness of reward models with transformed inputs. *arXiv preprint arXiv:2503.11751*, 2025.
- Yunjia Xi, Weiwen Liu, Yang Wang, Ruiming Tang, Weinan Zhang, Yue Zhu, Rui Zhang, and Yong Yu. On-device integrated re-ranking with heterogeneous behavior modeling. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5225–5236, 2023.
- Zhihao Xu, Yongqi Tong, Xin Zhang, Jun Zhou, and Xiting Wang. Understanding conflicts in multi-objective alignment through reward consistency. In *Findings of the Association for Computational Linguistics: ACL*, pp. 5450–5472, 2026.
- Yuzi Yan, Xingzhou Lou, Jialian Li, Yiping Zhang, Jian Xie, Chao Yu, Yu Wang, Dong Yan, and Yuan Shen. Reward-robust RLHF in LLMs. *arXiv preprint arXiv:2409.15360*, 2024.
- Rui Yang, Ruomeng Ding, Yong Lin, Huan Zhang, and Tong Zhang. Regularizing hidden states enables learning generalizable reward model for LLMs. In *Advances in Neural Information Processing Systems*, pp. 62279–62309, 2024.
- Chenlu Ye, Rui Yang, Quanquan Gu, and Tong Zhang. Corruption-robust offline reinforcement learning with general function approximation. In *Advances in Neural Information Processing Systems*, 2023.
- Jen-Yuan Yeh and Cheng-Jung Tsai. A graph-based feature selection method for learning to rank using spectral clustering for redundancy minimization and biased pagerank for relevance analysis. *Computer Science and Information Systems*, 19(1):141–164, 2022.

Yuanzhao Zhai, Han Zhang, Yu Lei, Yue Yu, Kele Xu, Dawei Feng, Bo Ding, and Huaimin Wang. Uncertainty-penalized reinforcement learning from human feedback with diverse reward LoRA ensembles. *arXiv preprint arXiv:2401.00243*, 2023.

Xiaoying Zhang, Jean-Francois Ton, Wei Shen, Hongning Wang, and Yang Liu. Overcoming reward overoptimization via adversarial policy optimization with lightweight uncertainty estimation. *arXiv preprint arXiv:2403.05171*, 2024.

Zhiwei Zhang, Hui Liu, Xiaomin Li, Zhenwei Dai, Jingying Zeng, Fali Wang, Minhua Lin, Ramraj Chandradevan, Zhen Li, Chen Luo, Xianfeng Tang, Qi He, and Suhang Wang. Bradley-Terry and multi-objective reward modeling are complementary. In *International Conference on Learning Representations*, 2026.

Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. Large language models are not robust multiple choice selectors. In *International Conference on Learning Representations*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, pp. 46595–46623, 2023.

Banghua Zhu, Michael I Jordan, and Jiantao Jiao. Iterative data smoothing: Mitigating reward overfitting and overoptimization in RLHF. *arXiv preprint arXiv:2401.16335*, 2024.

A RMB Details

A.1 Optimization of the Reward Model Boosting Objective

This section provides a detailed derivation for learning the tree aggregator, f_m , at each boosting iteration m . The primary goal is to determine the optimal tree structure and leaf values that minimize the overall objective function defined in Eqn. (3). The optimization follows the principle of XGBoost (Chen & Guestrin, 2016).

Objective function approximation. The training objective at the m -th iteration, as presented in Eqn. (3) of the main text, can be rewritten as the following form:

$$\mathcal{L}^{(k)} = \sum_{(x, y_w, y_l) \in \mathcal{D}} L(\Delta F_{k-1}(\mathbf{R}) + \Delta f_k(\mathbf{R})) + \Omega(f_k), \quad (10)$$

where $\Delta F_{k-1}(\mathbf{R}) = F_{k-1}(\mathbf{R}(x, y_w)) - F_{k-1}(\mathbf{R}(x, y_l))$ and $\Delta f_k(\mathbf{R}) = f_k(\mathbf{R}(x, y_w)) - f_k(\mathbf{R}(x, y_l))$. $\Omega(f_k)$ is a regularizer of a tree to penalize overly complicated trees structure and large leaf values. To facilitate efficient optimization, we approximate the loss function L using a second-order Taylor expansion around the prediction from the previous iteration, $\hat{y}^{(k-1)} = \Delta F_{k-1}(\mathbf{R})$, converting the problem into minimizing a quadratic form.

Let g_m and h_m be the first and second-order gradients of the loss function with respect to the prediction for the m -th preference data point in the training dataset, denoted by

$$g_m = \left. \frac{\partial L(y_m, \hat{y})}{\partial \hat{y}} \right|_{\hat{y}=\hat{y}_m^{(k-1)}} \quad (11)$$

$$h_m = \left. \frac{\partial^2 L(y_m, \hat{y})}{\partial \hat{y}^2} \right|_{\hat{y}=\hat{y}_m^{(k-1)}} \quad (12)$$

The loss function for a single instance can thus be approximated as:

$$L(\hat{y}_m^{(k-1)} + \Delta f_k(\mathbf{R})) \approx L(\hat{y}_m^{(k-1)}) + g_m \Delta f_k(\mathbf{R}) + \frac{1}{2} h_m \Delta f_k^2(\mathbf{R}). \quad (13)$$

After removing the constant term $L(\hat{y}_m^{(k-1)})$, we simplify the objective function to:

$$\tilde{\mathcal{L}}^{(k)} \approx \sum_{m=1}^M \left[g_m \Delta f_k(\mathbf{R}_m) + \frac{1}{2} h_m \Delta f_k^2(\mathbf{R}_m) \right] + \Omega(f_k). \quad (14)$$

Expressing the objective by the parameters of decision trees. A decision tree f_k is defined by its structure q and leaf values $\mathbf{w}$. The structure $q : \mathbb{R}^d \rightarrow \{1, \dots, T\}$ maps an input instance $\mathbf{R} \in \mathbb{R}^N$ to a specific leaf index in $1, \dots, T$, and w_j is the score assigned to the j th leaf node for $j = 1, \dots, T$.

To express the parameters explicitly in the objective in Eqn. 14, we regroup the sum over training instances to a sum over the disjoint sets of instances contained in each leaf. In particular, the term $\Delta f_k(\mathbf{R})$ can be expressed as the difference between the leaf values corresponding to the winning and losing responses: $\Delta f_k(\mathbf{R}) = w_{q(\mathbf{R}_w)} - w_{q(\mathbf{R}_l)}$. Let $I_j = \{k : q(\mathbf{R}_k) = j\}$ denote the set of instances partitioned into leaf j . By regrouping the summation in Eqn. (14) over the leaves, we obtain:

$$\tilde{\mathcal{L}}^{(k)} = \sum_{j=1}^T \left[\left(\sum_{m \in I_j} g_m \right) w_j + \frac{1}{2} \left(\sum_{m \in I_j} h_m \right) w_j^2 \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (15)$$

where the last two terms are the regularizer, i.e., $\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$. Let us define the sum of gradient statistics in each leaf as $G_j = \sum_{m \in I_j} g_m$ and $H_j = \sum_{m \in I_j} h_m$. The objective can be concisely written as:

$$\tilde{\mathcal{L}}^{(k)} = \sum_{j=1}^T \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T. \quad (16)$$

Optimal leaf weight calculation. The objective function in Eqn. (16) is a sum of independent quadratic functions of each leaf weight w_j . To find the optimal value w_j^* , we take the partial derivative of $\tilde{\mathcal{L}}^{(k)}$ with respect to w_j and set it to zero:

$$\frac{\partial \tilde{\mathcal{L}}^{(k)}}{\partial w_j} = G_j + (H_j + \lambda) w_j = 0, \quad (17)$$

which in turn yields the optimal leaf weight:

$$w_j^* = -\frac{G_j}{H_j + \lambda}. \quad (18)$$

Decision-tree structure learning. Finding the globally optimal tree structure is difficult, as it would require evaluating an exponential number of possible tree structures. Instead, we adopt a greedy, top-down approach to build the tree by making the best possible split at each step, which is computationally feasible and effective in practice.

We use *gain* in Eqn. (6) to quantify the quality of a potential split. Specifically, we first substitute the optimal leaf weight w_j^* in Eqn. (18) back into the objective function in Eqn. (16). This yields a score that evaluates the quality of a given tree structure q :

$$\tilde{\mathcal{L}}^{(m)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T. \quad (19)$$

Consider a split that partitions a parent leaf into two child leaves, left and right. The gain of this split is defined as the reduction in the objective function:

$$\text{Gain} = \tilde{\mathcal{L}}_{\text{parent}} - (\tilde{\mathcal{L}}_{\text{left_child}} + \tilde{\mathcal{L}}_{\text{right_child}}). \quad (20)$$

Combining Eqn. (19), we have

$$\text{Gain} = \frac{1}{2} \left[\underbrace{\frac{G_L^2}{H_L + \lambda}}_{\text{Left Score}} + \underbrace{\frac{G_R^2}{H_R + \lambda}}_{\text{Right Score}} - \underbrace{\frac{G^2}{H + \lambda}}_{\text{Parent Score}} \right] - \underbrace{\gamma T}_{\text{Penalty}}. \quad (21)$$

The algorithm enumerates all possible splits over all entries in $\mathbf{R}$ and selects the split that maximizes this gain. The process is repeated until a stopping criterion is met (i.e., reaching a maximum depth or observing no positive gain).

A.2 Details of the HSIC Diversity-Promoting Regularizer

In the §2.3, we aim to train a diverse ensemble of N reward models $\{R_{\theta_n}\}_{n=1}^N$. Specifically, we apply the Hilbert–Schmidt Independence Criterion (HSIC; Gretton et al., 2005; 2007) to the preference margins.

Consider a batch of b preference pairs $D_b = \{(x_w^{(i)}, x_l^{(i)})\}_{i=1}^b$ and two reward models R_{θ_m} and R_{θ_n} . We calculate the preference margin for the i th sample by

$$\mathbf{p}_m^{(i)} = R_{\theta_m}(x_w^{(i)}) - R_{\theta_m}(x_l^{(i)}), \quad (22)$$

$$\mathbf{p}_n^{(i)} = R_{\theta_n}(x_w^{(i)}) - R_{\theta_n}(x_l^{(i)}). \quad (23)$$

We group the preference margins of different samples as vectors, denoted by $\mathbf{p}_m$ and $\mathbf{p}_n$. HSIC assumes that the entries in $\mathbf{p}_m$ and $\mathbf{p}_n$ come from their respective underlying distributions, and the goal is to derive an empirical estimator for the dependence between the two distributions.

HSIC dependency measure. While the notion of covariance is a standard measure of dependency, it is restricted to linear correlation. The HSIC provides a more general way of capturing non-linear dependency by analyzing the relationship between variables in a high-dimensional feature space through kernel methods.

The HSIC begins by projecting the random variables p_m and p_n from their input space $\mathcal{X}$ onto a Reproducing Kernel Hilbert Space (RKHS), denoted $\mathcal{F}$, via a feature map $\phi(\cdot) : \mathcal{X} \rightarrow \mathcal{F}$. For an inner-product of $\phi(a)$ and $\phi(b)$, a computationally tractable kernel function is used

$$\langle \phi(a), \phi(b) \rangle_{\mathcal{F}} = k(a, b). \quad (24)$$

Thus, we do not need to explicitly compute $\phi(\cdot)$. A common choice for the kernel is the Gaussian RBF kernel, $k(a, b) = \exp\left(-\frac{\|a-b\|^2}{2\sigma^2}\right)$, where the hyperparameter σ controls its sensitivity.

Within this framework, the concept of covariance is generalized by the cross-covariance operator:

$$C_{mn} = \mathbb{E}_{p_m, p_n} [(\phi(p_m) - \mu_m) \otimes (\phi(p_n) - \mu_n)], \quad (25)$$

where $\mu_m = \mathbb{E}_{p_m}[\phi(p_m)]$ and $\mu_n = \mathbb{E}_{p_n}[\phi(p_n)]$ are the means. The tensor product $\otimes$ captures the complete covariance structure between the feature vectors.

HSIC is then defined as the squared Hilbert–Schmidt norm of this operator, which quantifies its magnitude:

$$\text{HSIC}(p_m, p_n) = \|C_{mn}\|_{HS}^2 \quad (26)$$

$$= \|\mathbb{E}_{p_m, p_n} [(\phi(p_m) - \mu_m) \otimes (\phi(p_n) - \mu_n)]\|_{HS}^2 \quad (27)$$

While the theoretical definition of HSIC is based on expectations over a distribution, in practice we must compute an empirical estimate from a finite set of data samples. Let $\{(p_m^{(i)}, p_n^{(i)})\}_{i=1}^b$ be b -many pairs of samples in a batch. The kernel matrices for the two variables are $\mathbf{K}_m \in \mathbb{R}^{b \times b}$ and $\mathbf{K}_n \in \mathbb{R}^{b \times b}$, given by

$$(\mathbf{K}_m)_{ij} = k(p_m^{(i)}, p_m^{(j)}), \quad (\mathbf{K}_n)_{ij} = k(p_n^{(i)}, p_n^{(j)}). \quad (28)$$

The empirical estimate for HSIC from this finite batch is given by a matrix formula:

$$\text{HSIC}_b(\mathbf{p}_m, \mathbf{p}_n) = \frac{1}{(b-1)^2} \text{tr}(\mathbf{K}_m \mathbf{H} \mathbf{K}_n \mathbf{H}), \quad (29)$$

where $\mathbf{H} \in \mathbb{R}^{b \times b}$ is the centering matrix, defined as

$$\mathbf{H} = \mathbf{I} - \frac{1}{b} \mathbf{1} \mathbf{1}^\top. \quad (30)$$

Here, $\mathbf{I}$ is the identity matrix and $\mathbf{1}$ is a column vector of ones. Overall, the HSIC serves as a regularizer in the reward model training objective in Eqn. (9), offering a computationally feasible way to capture non-linear dependency, and thus promotes diversity in our learned reward models.

Basic Setups	
Base models	Gemma-2b-it
Quantization for training	bf16
Fine-tuning strategy	LoRA
LoRA r	32
LoRA alpha	64
LoRA dropout	0.05
Optimizer	Adamw_hf
Batch size	16
Learning rate	1×10^{-5}
Learning rate scheduler	cosine
Warmup ratio	0.03
RMB Setups	
HSIC ratio λ	0.1
Max depth	5
Max round of boosting	256
L2 regularizer	0.1
PPO Setups (Schulman et al., 2017)	
KL regularization	0.0
Epochs	1
Learning rate	1×10^{-5}
Lambda for GAE	0.95
Gamma	1
Clip range	0.2
Number of optimization epochs per batch	4
Number of tokens during generation	512

Table 7: Key implementation details of our main experiments.

B Implementation Details

Reward model architecture. Base reward models use the *AutoModelForSequenceClassification* architecture from Wolf et al. (2020). The trainable parameters are a reward head and LoRA (Hu et al., 2022) adapters. The reward head is a two-layer feedforward network with 1024 dimensions and a ReLU activation function, which follows the setup in Yang et al. (2024). LoRA hyperparameters are listed in Tab. 7. All reward model learning approaches employ the same architectures and have the same trainable parameters.

Details of competing methods. We discuss the competing methods as follows.

- **Baseline.** This is a vanilla model trained to minimize the objective in Eqn. (7).
- **Training with labeled margin** (Wang et al., 2024a). This approach modifies the classic objective by enforcing that y_w exceeds y_l by a certain margin m :

$$\mathcal{L}_{\text{margin}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim D} [\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l) - m)]. \quad (31)$$

The Unified-Feedback dataset provides a margin label m for each pairwise sample, which indicates the desired reward difference. Note that such an approach requires additional labels (i.e., a margin label for every sample) compared with our method, and thus it may not be applicable to other preference datasets.

- **Label smoothing** (Wang et al., 2024a). To account for potential annotation noise, this approach assumes that a labeled pair is flipped with probability of ϵ (set to 0.1). Then the training objective

Reward model	Unified-Feedback	HHH-Align	MT-Bench	RewardBench				
				Avg.	Chat	Chat-Hard	Safety	Reasoning
Frozen RM	63.8	66.4	69.5	x	x	x	x	x
Baseline RM	72.1	73.4	71.2	68.2	95.5	38.0	73.8	65.3
Label Smooth	71.5	72.1	71.2	70.6	94.4	37.3	73.2	77.4
Margin Loss	72.0	75.0	72.6	70.2	95.8	38.4	73.9	72.5
GRM	73.2	79.8	73.4	70.8	97.8	42.1	77.9	65.2
Avg Ensemble ($N = 3$)	72.8	76.8	73.7	71.0	98.0	37.5	77.3	71.3
Avg Ensemble ($N = 5$)	73.2	79.5	73.8	71.6	95.3	43.4	78.3	71.4
RMB ($N = 3$)	74.4	81.8	75.2	72.0	95.6	44.1	78.1	71.8
RMB ($N = 5$)	75.1	82.6	75.6	72.2	91.9	39.4	80.7	73.7
RMB ($N = 8$)	75.9	83.0	76.6	73.3	95.1	43.5	76.6	74.5
RMB ($N = 10$)	74.9	82.1	76.2	72.6	96.9	42.8	77.9	73.3

Table 8: Preference prediction performance on all Reward Models learned using 400K data mode of Unified-Feedback. Results of competing approaches are directly sourced from Yang et al. (2024). N means the number of reward models used for ensembling or boosting.

thus becomes

$$\mathcal{L}_{\text{smooth}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim D} [(1 - \epsilon) \log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l)) + \epsilon \log \sigma(r_\theta(x, y_l) - r_\theta(x, y_w))], \quad (32)$$

This improves robustness to label errors and mitigates overfitting.

- **Avg Ensemble** (Coste et al., 2024). For this competing approach, we train three reward models (in the same way as the Baseline approach) with different random seeds and aggregate their outputs by simple averaging.
- **GRM** (Yang et al., 2024). GRM regularizes reward model learning with a text generation objective:

$$\mathcal{L}_{\text{GRM}}(\theta, \theta_{\text{LM}}) = -\mathbb{E}_{(x, y_w, y_l) \sim D} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))] - \alpha \mathbb{E}_{(x, y_c) \sim D} [\log \sigma(\beta \log(\pi_{\theta_{\text{LM}}}(y_c | x)))] \quad (33)$$

where we set $\beta = 0.1$ and $\alpha = 0.0005$ following the best setting reported by Yang et al. (2024).

Computing resources. Experiments are run on NVIDIA RTX A6000 (48 GB) GPUs. Training a 2B-parameter reward model with LoRA (Hu et al., 2022) on 40K training examples for two epochs requires approximately 31.9 GPU-hours.

C Additional Results

C.1 Results with a Different Data Size

Following the experimental setup of Yang et al. (2024), we trained our reward models on the Unified-Feedback dataset with both 40K and 400K samples, as described in §3.2. In the main paper, we only present the 40K-sample experiment (Tab. 1) due to the limit of space. In this appendix, we show the results of the 400K-sample experiment in Tab. 8. We see that our RMB model surpasses all competing methods on both in-distribution and out-of-distribution (OOD) datasets. These findings are consistent with the performance observed in the 40K-sample experiment (Tab. 1).

C.2 Results with Full Training Data and a Larger Backbone

We further evaluate RMB with a larger training set and a larger reward-model backbone. Specifically, we train all approaches from scratch on the full Unified-Feedback training set, which contains 848,574 preference examples. We first use Gemma-2B-IT as the backbone, following the setup of our main experiments. We then

Approach	Unified-Feedback	HHH-Align	MT-Bench	RewardBench
Baseline RM	71.9	74.3	72.8	68.3
Label Smoothing	72.1	71.2	71.4	72.2
Margin Loss	71.9	76.5	73.2	72.6
GRM	73.5	79.1	74.7	70.4
Avg Ensemble ($N = 3$)	73.6	77.7	73.4	71.8
RMB ($N = 3$)	75.8	80.5	75.9	73.0

Table 9: Preference prediction performance of all approaches trained on the full Unified-Feedback dataset using the Gemma-2B-IT backbone.

Approach	Unified-Feedback	HHH-Align	MT-Bench	RewardBench
Baseline RM	76.1	87.5	76.3	76.4
Label Smoothing	76.7	88.0	78.1	77.3
Margin Loss	75.3	86.8	76.0	77.2
GRM	76.9	87.6	77.0	78.6
Avg Ensemble ($N = 3$)	76.4	88.3	76.8	78.9
RMB ($N = 3$)	78.2	89.4	77.5	81.6

Table 10: Preference prediction performance of all approaches trained on the full Unified-Feedback dataset using the Mistral-7B backbone.

Approach	Opponent	Pairwise Preference Comparison (in %)				Pointwise Preference Comparison (in %)			
		Win $\uparrow$	Tie	Lose $\downarrow$	Δ	Win $\uparrow$	Tie	Lose $\downarrow$	Δ
RMB ($N = 3$)	Baseline RM	59.54	5.42	35.04	$\uparrow$ 24.50	47.28	20.19	32.53	$\uparrow$ 14.75
	Avg Ensemble ($N = 3$)	53.15	7.93	38.92	$\uparrow$ 14.23	39.40	30.91	29.69	$\uparrow$ 9.71
	GRM	57.38	9.37	33.25	$\uparrow$ 24.13	41.46	27.44	31.10	$\uparrow$ 10.36

Table 11: LLM-as-judge (GPT-4o) of policies learned via RLHF.

scale the reward-model backbone to Mistral-7B and repeat the same experiment. For both settings, we evaluate preference prediction performance on Unified-Feedback, HHH-Alignment, MT-Bench, and RewardBench.

As shown in Tabs. 9 and 10, RMB maintains strong performance when trained with the full Unified-Feedback dataset. With Gemma-2B-IT, RMB achieves the best performance across all four evaluation datasets. With Mistral-7B, RMB achieves the best performance on three of the four datasets and remains competitive on MT-Bench. These results show that the effectiveness of RMB holds with both substantially more training data and a larger reward-model backbone.

Computing resources. Training all approaches on the full Unified-Feedback dataset requires approximately 3,562 NVIDIA L40S GPU-hours with the Gemma-2B-IT backbone and 11,397 NVIDIA L40S GPU-hours with the Mistral-7B backbone.

C.3 Results of Avoiding LLM-as-Judge’s Inherent Biases

LLM may have its own stylistic preferences (e.g., verbosity, tone, formatting), which could introduce systematic bias in LLM-as-judge evaluation. To reduce this risk, we intentionally choose a judge from a different model family than the policy backbone (DeepSeek-R1 as judge vs. Gemma as generator) as shown in §3.2 and Tab. 2. This mitigates potential judge-generator coupling where the judge may implicitly favor outputs that resemble its own generation style or training priors, and thus helps avoid inflating the win rate due to family-specific alignment rather than reward quality. Moreover, all candidate policies are evaluated by the same judge under the same prompts and identical pairwise protocol, so any fixed judge bias is applied consistently across methods, and the comparison remains controlled.

To further check that the conclusion is not specific to DeepSeek-R1, we additionally evaluate with ChatGPT-4o as the judge using the exact same win/tie/lose methodology. The results in Tab. 11 confirm the same trend as Tab. 2, showing RMB consistently outperforms the baselines in policy training using different LLMs for evaluation.

C.4 Analysis of Robustness to Prompt-Dependent Reward Shifts

Our RM training uses the Bradley-Terry objective, which cannot identify the absolute reward values; i.e., the reward function may be shifted by a prompt-dependent constant. As mentioned in Section 6, our RMB does not guarantee preservation of preference order under such shifts. However, this is not a major threat to our method, because we train RMs first, and then train the boosting model given the RMs being fixed. Thus, the boosting model can adapt accordingly.

RM	Reward Mean	Reward Std.	$c(x)$ Mean	$c(x)$ Std.	$c(x)$ P05–P95
RM 0	2.6380	17.0647	-0.1391	17.0468	[-28.2891, 27.9354]
RM 1	0.6850	10.9031	0.0566	10.9953	[-17.9596, 18.3358]
RM 2	-0.0619	10.9822	0.0606	10.9897	[-17.8373, 17.7582]

Table 12: Statistics of reward scores and sampled prompt-dependent constants.

Dataset	Test Pairs	Rank Flips	Flip Ratio
Unified-Feedback	7,289	297	4.07%

Table 13: Sensitivity of RMB rankings to prompt-dependent additive shifts.

Nevertheless, we examine the sensitivity of RMB when such a constant is deliberately introduced at the inference stage after training the boosting model.

Mimicking prompt-dependent constants. We instantiate the prompt-dependent constants by independently sampling from a Gaussian distribution. To make the perturbation scale comparable with the raw scores of each RM, we consider the standard deviation of our trained RMs. Specifically, for RM i , we compute σ_i over all preferred and disfavored responses in the test set. Then, the prompt-dependent constants are sampled

$$c_i(x) \sim \mathcal{N}(0, \sigma_i^2), \quad \mathbf{C}(x) = [c_1(x), c_2(x), c_3(x)]. \quad (34)$$

Then, we apply the same $\mathbf{C}(x)$ to the preferred and disfavored responses:

$$\mathbf{R}'(x, y_w) = \mathbf{R}(x, y_w) + \mathbf{C}(x), \quad \mathbf{R}'(x, y_l) = \mathbf{R}(x, y_l) + \mathbf{C}(x). \quad (35)$$

With such a shift, we test the consistency of preference order of our RMB by using $\mathbf{R}$ or $\mathbf{R}'$.

Experiments. We conduct the experiment on the `llm-blender/Unified-Feedback` test set containing 7,289 preference pairs. In the experiment, the XGBoost aggregator is trained on the original RMs and is fixed when we add the sampled constants to RMs. We show the statistics of reward scores and the sampled constants in Tab. 12.

We show the number and ratio of flipped pairwise order in Tab. 13. As seen, only 297 out of 7,289 preference orders are changed in the RMB ranking, corresponding to a rank flip ratio of 4.07%. This indicates that RMB’s rankings remain largely stable if RMs are shifted.

D Training Efficiency

This appendix presents a comprehensive analysis of the computational efficiency of RMB, detailing the breakdown of training costs and examining the trade-off between performance and efficiency as the number of reward models, N , increases. We demonstrate that RMB incurs a training cost comparable to existing ensemble-based approaches and that the method remains robust across reasonable choices of N .

Reward-Model Training Cost The primary computational overhead in RMB stems from training the base RMs. As the number of RMs increases, the training cost grows linearly. In our experimental setup, training a single RM per epoch of data requires approximately 31.9 GPU-hours on an NVIDIA RTX A6000 (48GB). However, the most appropriate comparison is not against a single, weak RM, but rather against other methods that employ multiple RMs to mitigate reward hacking, such as reward model ensembles (Coste et al., 2024; Eisenstein et al., 2024). When compared under a fixed budget of N trained RMs, RMB proves to be both efficient and effective. For example, with $N = 3$, RMB outperforms an average ensemble using $N = 3$ or even $N = 5$ on both preference prediction and downstream RLHF tasks. This indicates that RMB achieves superior performance with lower total computational expenditure than competing ensemble-based methods, which often require training a larger number of models to achieve similar or weaker gains.

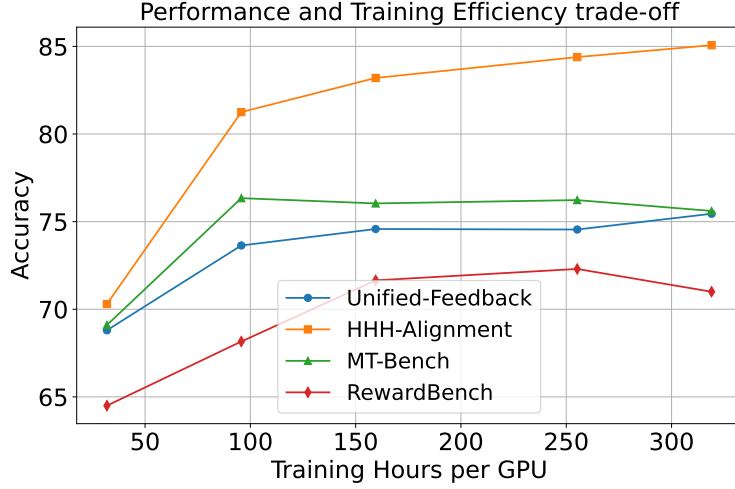


Figure 5: Performance–training efficiency trade-off.

Cost of HSIC Diversity Regularization RMB jointly trains N RMs with a diversity-promoting regularizer based on the HSIC. This term is computed over per-batch margin vectors and is implemented via a small number of matrix operations. In practice, the computational overhead of this regularization is negligible relative to the forward and backward passes required for the N large language models serving as RMs.

Boosting Aggregator Training Cost Following the training of the RMs, RMB learns a tree-based boosting aggregator on the frozen RM outputs. This stage is extremely lightweight; in our experiments, training the aggregator on the full dataset requires less than 5 minutes on a CPU. This is negligible compared to the GPU-hours required for RM training. Consequently, the total training cost of RMB remains on the same order as existing ensemble methods while delivering substantially higher accuracy and robustness.

Performance–Training Efficiency Trade-Off Fig. 5 illustrates the preference prediction accuracy against the total training cost (in GPU-hours per GPU) for varying ensemble sizes $N \in \{1, 3, 5, 8, 10\}$. Several key trends emerge from this analysis. First, we observe monotonic improvements for small N ; increasing N from 1 to 5 yields consistent accuracy gains across both in-distribution and out-of-distribution evaluation sets, with training costs growing roughly linearly. Second, performance gains begin to saturate and exhibit mild overfitting as N increases from 8 to 10. This plateau suggests that as the input dimension (the vector of N RM scores) grows, an overly flexible aggregator may begin to fit spurious correlations or idiosyncratic noise within the specific collection of RMs rather than learning a generalizable aggregation rule, which suggests overfitting due to an over-powerful aggregator model. Finally, the performance curve reveals a wide and relatively flat plateau for $N \in [5, 10]$. This stability indicates that RMB is not sensitive to precise tuning of N , as a broad range of values yields similarly strong results without sharp performance drops.

E Choosing the Number of Reward Models N

This appendix provides a principled perspective on selecting the ensemble size N in RMB. Increasing N can improve robustness by aggregating information across multiple reward models, but the gains diminish when additional models contribute little *independent* signal. Moreover, a larger ensemble increases the input dimensionality of the downstream boosting aggregator, which can raise the risk of overfitting. We formalize these effects using a standard correlated-error model, derive an *effective ensemble size* that makes saturation explicit.

E.1 A Correlated-Error Model for Ensemble Margins

To quantify diminishing returns with increasing N , we adopt a classical exchangeable (equicorrelated) error model Krogh & Vedelsby (1994); Donner & Klar (2000); Bühlmann & Yu (2002); Kuncheva (2014). Write each margin predictor in Eqn. (8) as

$$m_k(x) = m^*(x) + \varepsilon_k(x), \quad (36)$$

where $m^*(x)$ is a latent margin signal and $\varepsilon_k(x)$ is the model-specific error. For tractability, we assume that for any fixed x :

$$\mathbb{E}[\varepsilon_k(x)] = 0, \quad (37)$$

$$\text{Var}(\varepsilon_k(x)) = \sigma^2 \quad \text{for all } k, \quad (38)$$

$$\text{Corr}(\varepsilon_i(x), \varepsilon_j(x)) = \rho \quad \text{for all } i \neq j. \quad (39)$$

E.2 Variance of the Averaged Margin and Diminishing Returns

Define the averaged margin vector over N RMs:

$$\bar{m}_N(x) \triangleq \frac{1}{N} \sum_{k=1}^N m_k(x) = m^*(x) + \bar{\varepsilon}_N(x), \quad \bar{\varepsilon}_N(x) \triangleq \frac{1}{N} \sum_{k=1}^N \varepsilon_k(x). \quad (40)$$

Since $m^*(x)$ is fixed given x , the variance of $\bar{m}_N(x)$ is the variance of $\bar{\varepsilon}_N(x)$. We compute $\text{Var}(\bar{\varepsilon}_N)$ explicitly:

$$\begin{aligned} \text{Var}(\bar{\varepsilon}_N) &= \text{Var}\left(\frac{1}{N} \sum_{k=1}^N \varepsilon_k\right) = \frac{1}{N^2} \text{Var}\left(\sum_{k=1}^N \varepsilon_k\right) \\ &= \frac{1}{N^2} \left(\sum_{k=1}^N \text{Var}(\varepsilon_k) + 2 \sum_{1 \leq i < j \leq N} \text{Cov}(\varepsilon_i, \varepsilon_j) \right) \\ &= \frac{1}{N^2} \left(N\sigma^2 + 2 \binom{N}{2} \rho\sigma^2 \right) = \sigma^2 \frac{1 + (N-1)\rho}{N}. \end{aligned} \quad (41)$$

An equivalent and often more interpretable form is

$$\text{Var}(\bar{\varepsilon}_N) = \sigma^2 \left(\rho + \frac{1-\rho}{N} \right). \quad (42)$$

Eqn. (42) highlights two regimes. If $\rho \approx 0$ (nearly independent errors), the variance decays as σ^2/N . If $\rho > 0$, the variance approaches the non-vanishing floor $\sigma^2\rho$ as $N \rightarrow \infty$, and gains from increasing N must eventually saturate.

E.3 Effective Ensemble Size N_{eff}

A useful summary of the correlation penalty is the *effective ensemble size* N_{eff} , defined as the size of an *independent* ensemble ($\rho = 0$) that would achieve the same variance as the correlated ensemble. For independent errors, $\text{Var}(\bar{\varepsilon}) = \sigma^2/N_{\text{eff}}$. Equating this to Eqn. (41) yields

$$\frac{\sigma^2}{N_{\text{eff}}} = \sigma^2 \frac{1 + (N-1)\rho}{N} \implies N_{\text{eff}} = \frac{N}{1 + (N-1)\rho}. \quad (43)$$

This expression makes the dependence-limited regime explicit: when $\rho > 0$, N_{eff} grows sublinearly and satisfies $N_{\text{eff}} \rightarrow 1/\rho$ as $N \rightarrow \infty$. In other words, adding more reward models cannot yield unbounded benefits unless their errors are close to independent.

E.4 A Guideline for Selecting N

Eqn. (42) shows that the reducible portion of the averaged-error variance is

$$\text{Var}(\bar{\varepsilon}_N) - \sigma^2 \rho = \sigma^2 \frac{1 - \rho}{N}. \quad (44)$$

This motivates selecting N so that the remaining reducible term is below a user-chosen tolerance. For a tolerance $\tau > 0$,

$$\sigma^2 \frac{1 - \rho}{N} \leq \tau \implies N \geq \frac{\sigma^2(1 - \rho)}{\tau}. \quad (45)$$

Because σ^2 is a scale factor, a convenient alternative is a *relative* tolerance $\eta \in (0, 1)$ defined with respect to the reducible term at $N = 1$:

$$\frac{\text{Var}(\bar{\varepsilon}_N) - \sigma^2 \rho}{\text{Var}(\bar{\varepsilon}_1) - \sigma^2 \rho} = \frac{1}{N} \leq \eta \implies N \geq \frac{1}{\eta}. \quad (46)$$

This makes the diminishing-returns phenomenon explicit: once N is moderately large, further increases can only shrink the remaining reducible term by a factor proportional to $1/N$, while the correlation floor $\sigma^2 \rho$ is unchanged.

E.5 Estimating Dependence on Held-out Data

In practice, ρ is unknown but can be approximated using held-out margins. Let $\mathcal{D}_{\text{val}}$ be a validation set of preference pairs. For each model k , form the margin vector

$$\mathbf{m}_k = (m_k(x))_{x \in \mathcal{D}_{\text{val}}} \in \mathbb{R}^{|\mathcal{D}_{\text{val}}|}. \quad (47)$$

A simple proxy for inter-model dependence is the average pairwise correlation of centered margin vectors:

$$\hat{\rho} \triangleq \frac{2}{N(N-1)} \sum_{i < j} \text{Corr}(\mathbf{m}_i - \bar{\mathbf{m}}, \mathbf{m}_j - \bar{\mathbf{m}}), \quad \bar{\mathbf{m}} \triangleq \frac{1}{N} \sum_{k=1}^N \mathbf{m}_k. \quad (48)$$

This estimate focuses on shared variation across examples rather than shared offsets. Using $\hat{\rho}$, one can compute $\hat{N}_{\text{eff}}(N) = \frac{N}{1 + (N-1)\hat{\rho}}$ as a diagnostic: a plateau in $\hat{N}_{\text{eff}}$ indicates that additional reward models are largely redundant.

E.6 Interaction with the Boosting Aggregator and a Practical Selection Procedure

The analysis above quantifies variance reduction under averaging. RMB uses a learned boosting aggregator rather than a simple mean, which makes validation-based selection particularly important. As N increases, the aggregator receives more features and may fit spurious interactions among partially redundant reward-model outputs, even with regularization. Therefore, we recommend selecting N using a two-criterion procedure that mirrors the theory:

(1) Diversity criterion (dependence-limited). Increase N only while $\hat{N}_{\text{eff}}(N)$ continues to grow meaningfully. Stop when

$$\frac{\hat{N}_{\text{eff}}(N+1) - \hat{N}_{\text{eff}}(N)}{\hat{N}_{\text{eff}}(N)} < \delta, \quad (49)$$

for a small $\delta \in [0.01, 0.05]$.

(2) Generalization criterion (complexity-limited). For each candidate N , train the boosting aggregator with early stopping on $\mathcal{D}_{\text{val}}$ using the same pairwise objective as in RMB. Increase N only if the *validation* objective improves by at least ϵ (or if the improvement is statistically reliable under bootstrap resampling over validation examples).

This procedure is computationally efficient in RMB because reward-model training typically dominates cost and scales approximately linearly with N . In practice, one can train reward models incrementally (e.g., add 1–2 models at a time), cache their margins on $\mathcal{D}_{\text{val}}$, and re-fit only the lightweight boosting aggregator during selection.

F HSIC Ratio Analysis

The selection of the diversity regularization coefficient, λ_{HSIC} , follows the established practice of balancing the main task loss with the diversity penalty (Dong et al., 2023; Li et al., 2024). In this appendix, we perform a sensitivity analysis by sweeping λ_{HSIC} across four orders of magnitude (from 0.001 to 10) to evaluate its impact on model performance. We report results on the Unified-Feedback dataset, utilizing the same experimental setup detailed in Section 3.4.

λ_{HSIC}	Avg. Indiv. RM Perf.	Avg. Pairwise Corr.	RMB Perf.
0.001	70.7 ± 0.8	92%	71.7
0.01	71.3 ± 1.1	86%	72.8
0.1	71.1 ± 2.5	73%	74.6
1	68.8 ± 3.0	59%	71.2
10	61.7 ± 3.7	51%	66.5

Table 14: Sensitivity analysis of the λ_{HSIC} . We report the average performance of individual Reward Models, the average pairwise correlation between them, and the final RMB performance.

As presented in Tab. 14, our approach demonstrates strong robustness to the choice of λ_{HSIC} . The performance of RMB remains stable and effective across a broad range of values ($0.001 \leq \lambda_{\text{HSIC}} \leq 1$), even as the average pairwise correlation among reward models varies significantly (from 92% down to 59%). This indicates that RMB can effectively leverage increasing diversity to boost performance, provided the base estimators maintain reasonable accuracy. Performance deterioration is observed only when the regularization becomes overwhelmingly dominant ($\lambda_{\text{HSIC}} = 10$). In this regime, while diversity is maximized (correlation drops to 51%), the aggressive penalty interferes with the optimization of the main reward objective. Consequently, the accuracy of individual reward models degrades sharply (61.7%), which in turn limits the effectiveness of the boosted aggregator.

G Diversity Regularizer Selection

In this appendix, we present an additional analysis for comparing the HSIC regularizer with two other widely used diversity-promoting objectives: Negative Correlation Learning (NCL) (Liu & Yao, 1999) and Determinantal Point Processes (DPP) (Kulesza & Taskar, 2012).

We prioritize HSIC as the diversity-promoting regularizer in the reward model learning scenario for two reasons. First, HSIC allows for direct control of statistical dependence by measuring and penalizing the dependence between the margin vectors of different RMs. This is precisely the form of diversity required for effective boosting-based aggregation. Second, the HSIC objective is bounded and scale-stable. The empirical HSIC values are strictly bounded between 0 and 1, ensuring a well-behaved scale when combined with the primary reward-learning objective. In contrast, losses based on DPP or NCL are theoretically unbounded from above, which complicates the selection of trade-off coefficients and can destabilize the optimization process.

We conducted a pilot study where we replaced the HSIC term with either DPP or NCL while keeping all other settings identical for $N = 5$ RMs. Tab. 15 reports the resulting preference prediction accuracies on the Unified-Feedback dataset.

The results indicate that both DPP and NCL yield substantially lower accuracies across all five RMs. Furthermore, they proved more sensitive to hyperparameter tuning due to the unbounded magnitude of their loss terms. By contrast, HSIC facilitates the training of individually superior RMs while maintaining stable

Regularizer	RM1	RM2	RM3	RM4	RM5
HSIC	73.40	70.73	69.29	68.05	73.84
DPP	60.58	64.93	55.20	62.74	57.19
NCL	56.02	59.45	52.87	57.40	55.16

Table 15: Comparison of different diversity regularizers. We report the accuracy (%) of five individually trained reward models (RM1–RM5) under HSIC, DPP, and NCL regularization. HSIC consistently yields stronger RMs, while DPP and NCL significantly underperform.

Please act as an impartial judge and evaluate the quality of the responses provided by two AI assistants to the user question displayed below. Your evaluation should consider correctness and helpfulness. You will be given a reference answer, assistant A’s answer, and assistant B’s answer. Your job is to evaluate which assistant’s answer is better. Begin your evaluation by comparing both assistants’ answers with the reference answer. Identify and correct any mistakes. Avoid any position biases and ensure that the order in which the responses were presented does not influence your decision. Do not allow the length of the responses to influence your evaluation. Do not favor certain names of the assistants. Be as objective as possible.

Source: [Source]
 Answer [ID1]: [Answer-A]
 Answer [ID2]: [Answer-B]

After providing your explanation, output your final verdict by strictly following this format: "[[ID1]]" if assistant A is better, "[[ID2]]" if assistant B is better, and "[[ID-TIE]]" for a tie. Your response should use the format:

Overall Quality: one-sentence comparison and explanation
 Preferred: [[Answer ID]]

Table 16: Prompt templates for LLM evaluation on text generated by RLHF optimized policy in §3.2 from a pairwise view. This template originates from Zheng et al. (2023)’s work, and is adapted according to Fan et al. (2026)’s template to avoid ID and position bias, aiming to provide reliable LLM judges for pairwise comparison.

optimization dynamics. Based on these empirical findings, we adopt HSIC as the diversity regularizer for RMB and exclude DPP and NCL from the main model design.

H LLM Evaluation Prompt Template

To evaluate the quality of the text generated by policies trained with different reward signals, we employ the LLM-as-judge paradigm, which has been widely adopted for automated evaluation of generative models (Zheng et al., 2023; Liu et al., 2024; Fan et al., 2026). In particular, we prompt Deepseek-R1 (DeepSeek-AI Team, 2025) to conduct both pairwise and pointwise assessments of the generated responses. Our prompt templates originate from Zheng et al. (2023), and are shown in Tab. 16 and 17. To ensure the fairness and reliability of our pairwise comparisons, we adapt the prompt structure in Fan et al. (2026), which is designed to mitigate ID and position bias (Zheng et al., 2024; Shen et al., 2023).

I Reproducibility Statement

All code is provided via a GitHub repository, including implementations for data loading, reward model training, and policy optimization. The datasets used are publicly available, and we release the complete set of training hyperparameters. Our evaluation approaches are also publicly available and can be fully reproduced.

Please act as an impartial judge and evaluate the quality of the response provided by an AI assistant to the user's question shown below. Your evaluation should consider factors such as helpfulness, relevance, accuracy, depth, creativity, and level of detail in the response. Begin your evaluation with a short explanation. Be as objective as possible. After providing your explanation, please rate the response on a scale of 1 to 10 by strictly following this format: "[rating]", for example: "Rating: [[5]]".

[Question]
{question}
[The Start of Assistant's Answer]
{answer}
[The End of Assistant's Answer]

Table 17: Prompt templates for LLM evaluation on text generated by RLHF optimized policy in §3.2 from a pointwise view. This template originates from Zheng et al. (2023)'s work, aiming to provide LLM judges for single-answer grading.